GhostLeaks Enterprise Edition

Complete Documentation — A to Z

August 2026

- 0. About This Document
- 1. Overview (README)
- GhostLeaks Enterprise Edition
  - Overview
  - Feature List
  - Screenshots
  - Architecture Overview
  - Technology Stack
  - Folder Structure
  - Installation
  - Environment Variables
  - Deployment
  - Domain Monitoring Workflow
  - Security Highlights
  - REST API Overview
  - Webhook Overview
  - Multi-Tenancy
  - White-Label Customization
  - Demo Mode
  - Public Marketing Pages
  - Buyer Benefits
  - Business Use Cases
  - FAQ
  - Documentation Index
- 2. Feature Guide
- Feature Guide
  - Who This Is For — and What You Save
  - 1. Accounts, Workspaces, and Tenants
  - 2. Email Breach Check
  - 3. Password Breach Check
  - 4. Bulk CSV Scan
  - 5. Domain Monitoring
  - 6. Findings
  - 7. Exposure Timeline
  - 8. Executive Risk Score
  - 9. PDF Exposure Reports
  - 10. White-Label Branding
  - 11. Demo Mode
  - 12. Audit Log
  - 13. REST API (X-API-Key)
  - 14. Webhooks (Outbound)
  - 15. Scheduled Breach Alerts
  - 16. Billing (Stripe)
  - 17. Self-Service Data Export (GDPR)
  - 18. Self-Service Account Deletion (GDPR)
  - 19. Multi-Provider Breach Intelligence
  - Where to go next
- 3. System Architecture

Overview
1. Technology Stack
2. Request Lifecycle
3. Data Model
4. Multi-Tenancy and Isolation
5. The Breach-Checking Pipeline
6. Authentication and API Security
7. Rate Limiting
8. Payment Architecture
9. Known Architectural Boundaries
4. REST API Reference
API Reference
Authentication
Rate Limits
Endpoints
Webhooks
Internal Endpoints (session-authenticated, not part of the public integration API)
Versioning
5. Security
Security Policy
Reporting a Vulnerability
Built-in Security Measures
Known Limitations
Dependency and Third-Party Risk
6. Deployment Guide
Deployment Guide
0. First hour — plain-language checklist
1. Option A — Docker Compose (recommended)
2. Option B — Manual host deployment
3. Environment Variables
4. Upgrading an existing deployment
5. Domain Monitoring — DNS Setup
6. Scheduled Breach Alerts
7. Multi-Tenant / White-Label Operational Notes
8. Backups
9. Demo Mode
7. Frequently Asked Questions
Frequently Asked Questions
Licensing
Deployment
Breach Data and Providers
Data and Compliance
Support
8. Selling and Positioning This Product
Buyer Guide
What you are actually getting
Before you sell anything
Positioning by buyer type
What this product does not include
Getting Help
9. License Summary
GhostLeaks Enterprise Edition — Commercial License Agreement
1. Grant of License
2. Restrictions
3. Intellectual Property
4. Third-Party Services
5. Disclaimer of Warranty

6. Limitation of Liability
7. Updates and Support
8. Transfer and Assignment
9. Termination
10. Governing Law
11. Entire Agreement
Contact
10. Version History
Changelog
[1.0.2] — 2026-08-12
[1.0.1] — 2026-08-05
[1.0.0] — 2026-08-04
[2.7.0] — 2026-08-02
[2.6.2] — 2026-08-01
[2.6.1] — 2026-07-29
[2.6.0] — 2026-07-28
[2.5.0] — 2026-07-28
[2.4.0] — 2026-07-28
[2.3.0] — 2026-07-26
[2.2.0] — 2026-07-25
[2.1.0] — 2026-07-24
[2.0.0] — 2026-07-22
Earlier history

0. About This Document

This is the complete documentation set for GhostLeaks Enterprise Edition, combined into a single reference: overview, every feature explained, system architecture, the REST API, security design, deployment instructions, frequently asked questions, licensing terms, and the full version history.

A separate, standalone **System Architecture** document covers the technical internals in more depth if you only need that part.

-e

1. Overview (README)

GhostLeaks Enterprise Edition

Self-hosted, white-label breach monitoring for MSPs, security agencies, and internal security teams.

Deployment · API Reference · Security · Architecture · Buyer Guide · FAQ

Overview

GhostLeaks Enterprise Edition is a self-hosted SaaS platform that monitors emails, passwords, and company domains for exposure in known data breaches. It is sold as commercial source code: you deploy it under your own brand, connect your own breach-intelligence provider(s), and run it on your own infrastructure or your customers'.

It is built for:

- **MSPs and cybersecurity agencies** offering breach monitoring as a managed service to clients, under their own name.
- **Software agencies and SaaS companies** that want to embed breach monitoring as a feature without building the provider integrations, scoring logic, or multi-tenancy from scratch.
- **Internal security teams** monitoring their own organization's domain and staff accounts.

This document is the top-level overview. For deployment, see `DEPLOYMENT.md`; for what you're licensing, see `LICENSE.md`; for a buyer-oriented walkthrough, see `BUYER_GUIDE.md`.

Feature List

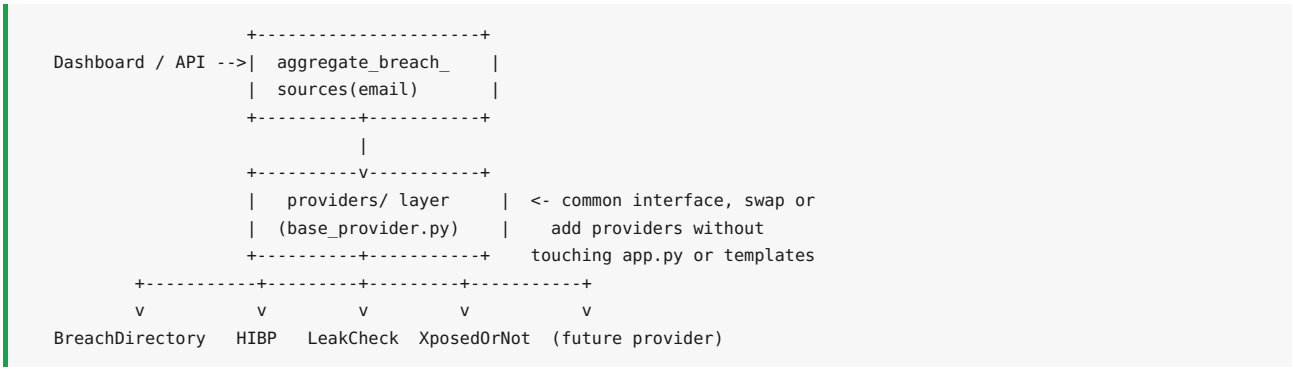
Feature	Description
Email breach monitoring	Checks a given email against configured breach-intelligence providers.
Password breach check (k-Anonymity)	Checks a password against known-breached password hashes without the full password ever leaving the client.
Bulk CSV email scanning	Upload up to 100 emails at once and receive a per-email result.
Domain monitoring with DNS ownership verification	A domain must be proven via a DNS TXT record before any scan can run against it.
Employee email monitoring	Once a domain is verified, individual employee emails can be added and are scanned one by one — see Domain Monitoring Workflow.
Executive Risk Score	A single 0-100 score per account, derived from breach count, source correlation, and exposure recency.
Executive PDF reports	Downloadable PDF summary for an individual scan or bulk scan result.
White-label branding	Per-tenant brand name, primary color, and logo, applied to the dashboard, timeline, and PDF reports.
Demo Mode	Workspace owners can populate their tenant with realistic sample data (users, a verified domain, findings, scan history, audit log) for demos and evaluation, and remove it cleanly afterward — see Demo Mode.
Multi-tenant architecture	Each customer/workspace is isolated at the data layer by `tenant_id`.
Role-based access control	`owner`, `admin`, and `member` roles. The owner can promote a member to admin or demote them back, from Workspace Settings.
REST API	Programmatic access to email checks, password checks, and risk score, authenticated by a per-user API key.
Webhooks	A per-user webhook URL is called when a new breach is detected for that user.
Scheduled monitoring	A protected HTTP endpoint re-checks all paid, verified users and emails them about new exposures — see the note in Deployment.
Audit logs	Admin-visible log of account and workspace-level actions.

SQLite and PostgreSQL support	SQLite works out of the box for evaluation; PostgreSQL is recommended for production.
Bring-your-own database	Point `DATABASE_URL` at your own managed database — no code changes required.
Bring-your-own API keys	Every breach-data provider, payment gateway, and email provider key is supplied by you via environment variables.
Multiple breach-intelligence providers	BreachDirectory, HIBP, LeakCheck, XposedOrNot — selectable and combinable via configuration.
Provider-independent architecture	Adding a new breach data source requires one new provider module — no changes to routes, templates, or existing providers.

Screenshots

Landing page	Dashboard	Domain monitoring
`docs/screenshots/landing.png`	`docs/screenshots/dashboard.png`	`docs/screenshots/domains.png`

Architecture Overview



Every breach-data provider implements the same `check_email()` contract. Which provider(s) are active is controlled entirely by the `BREACH_PROVIDER` environment variable — no code changes required to switch providers or run more than one for cross-confirmation. See `ARCHITECTURE.md` for full design rationale, including the security boundary that providers may only return the *fact* of exposure (source name, confidence, date) and never a credential value.

Supported Providers

Provider	Status	Notes
BreachDirectory (via RapidAPI)	Supported	Default provider.
Have I Been Pwned (HIBP)	Supported	Requires a paid HIBP API subscription.
LeakCheck	Supported	
XposedOrNot	Supported	Free tier works without an API key.

Technology Stack

- **Backend:** Flask, SQLAlchemy, Flask-Login, Flask-Bcrypt, Flask-Limiter, Flask-WTF (CSRF)
- **Database:** PostgreSQL (recommended for production) or SQLite (default, zero-config)

- **Frontend:** Server-rendered Jinja2 templates, no build step, one shared CSS design system
- **PDF generation:** ReportLab
- **Transactional email:** Brevo (Sendinblue) API
- **Payments:** Razorpay
- **Process manager:** Gunicorn

Folder Structure

```
app.py                    # Routes, models, and business logic
providers/
  base_provider.py        # Interface every breach-data provider implements
  breachdirectory.py
  hibp.py
  leakcheck.py
  xposedornot.py
templates/                 # Server-rendered pages (Jinja2)
  partials/
    footer.html            # Shared footer, included on every public page
static/
  gl-design-system.css     # Shared design tokens and components
  favicon.svg
ARCHITECTURE.md
DEPLOYMENT.md
BUYER_GUIDE.md
API.md
SECURITY.md
CHANGELOG.md
SUPPORT.md
ROADMAP.md
FAQ.md
LICENSE.md
CONTRIBUTING.md
CODE_OF_CONDUCT.md
Dockerfile
docker-compose.yml
.env.example
gunicorn_config.py
docker-entrypoint.sh
requirements.txt
```

Installation

```
        pip install -r requirements.txt
        cp .env.example .env
        # fill in .env -- see Environment Variables below
        python3 app.py
```

On first run, the application creates all required database tables automatically.

Environment Variables

Variable	Purpose
`SECRET_KEY`	Flask session/token signing. Required in production.
`DATABASE_URL`	Defaults to a local SQLite file if unset. Set to a PostgreSQL URL for production.
`BREACH_PROVIDER`	Active provider(s) — e.g. `breachdirectory` or `breachdirectory,hibp`.

`RAPIDAPI_KEY`	BreachDirectory (via RapidAPI).
`HIBP_API_KEY`	Have I Been Pwned.
`LEAKCHECK_API_KEY`	LeakCheck.
`XPOSEDORNOT_API_KEY`	XposedOrNot (optional).
`RAZORPAY_KEY_ID` / `RAZORPAY_KEY_SECRET`	Payment processing.
`BREVO_API_KEY` / `MAIL_USERNAME`	Transactional email (verification, password reset, breach alerts).
`APP_PUBLIC_URL`	Public URL of your deployment, used in emails and reports.
`DEFAULT_BRAND_NAME` / `DEFAULT_BRAND_COLOR` / `DEFAULT_BRAND_LOGO`	Branding shown on pages before anyone logs in (landing, login, register) — see White-Label Customization.
`ALERT_SECRET`	Shared secret required to call the scheduled breach-alert endpoint.

Deployment

Full instructions are in `DEPLOYMENT.md`. Summary:

- Runs behind Gunicorn in production (`gunicorn -c gunicorn_config.py app:app`).
- Works with SQLite out of the box for evaluation; use a managed PostgreSQL database for production.
- **Scheduled monitoring requires an external scheduler.** The application does not run its own background job scheduler — `GET /send-breach-alerts` (protected by the `X-Alert-Secret` header) must be called periodically by something outside the app itself: your host's cron/scheduled-job feature, a cron container, or an external uptime/cron service.
- Docker deployment is supported: `Dockerfile` , `docker-compose.yml` , and `.env.example` are included in the project root. `docker compose up -d` builds the app image, starts a bundled PostgreSQL container, waits for it to be healthy, then starts the app.

Domain Monitoring Workflow

1. **Add a domain.** An admin-level user (see Security for current role definitions) adds a company domain. A random verification token is generated.
2. **Verify ownership.** The workspace publishes a DNS TXT record at `_ghostleaks-verify.<domain>` with the value `ghostleaks-verify=<token>` . GhostLeaks performs a live DNS lookup; the domain is only marked verified on a match.
3. **Add employee emails.** Once verified, individual employee emails can be added to the domain.
4. **Scan.** Each tracked employee email is checked individually against the configured provider(s) and results are rolled up under the domain. If no employee emails have been added yet, the scan falls back to checking the bare domain name as a search term — breach providers index individual emails rather than domain names, so this fallback rarely returns meaningful results and exists only as a safety net, not as the primary mechanism.

Security Highlights

- Passwords are checked using k-Anonymity; the full password is never transmitted.
- Breach provider responses are filtered before reaching any template or API response — only exposure metadata (source, confidence, status) is ever surfaced, never a credential value.
- Domain-wide scans require DNS-verified ownership before they can run.
- CSRF protection is applied to state-changing forms.

Full detail, including current gaps and known limitations, is in `SECURITY.md`.

REST API Overview

Authenticated via a per-user API key sent as the `X-API-Key` header.

Method	Endpoint	Purpose
`POST`	`/api/v1/check-email`	Check a single email against configured providers.
`POST`	`/api/v1/check-password`	Check a password via k-Anonymity.
`GET`	`/api/v1/risk-score`	Retrieve the authenticated user's current risk score.

Full request/response examples are in `API.md`.

Webhook Overview

Each user may configure one webhook URL (in Settings). When a new breach is detected for that user via the API or the scheduled alert job, GhostLeaks sends a JSON payload to that URL. See `API.md` for the payload shape.

Multi-Tenancy

Every customer-facing record (users, domains, findings, scan history) is scoped by `tenant_id`. Registering without an invite link creates a new, isolated tenant with that user as `owner`; registering via a workspace's invite link (`/register?tenant_slug=...`) joins that existing tenant as a `member`. There is no cross-tenant data access at the query layer.

White-Label Customization

Each tenant can set a brand name, primary color, and logo URL from the Workspace settings page. These values are applied across the dashboard, timeline, and PDF reports for that tenant only — other tenants on the same deployment are unaffected.

Pages that exist before anyone logs in — the landing page, login, register, pricing, and the other public pages — aren't tied to any single tenant, so per-tenant branding doesn't apply to them. These use `DEFAULT_BRAND_NAME`, `DEFAULT_BRAND_COLOR`, and `DEFAULT_BRAND_LOGO` instead (see Environment Variables). If a deployment is dedicated to a single company rather than run as multi-tenant SaaS, setting these three variables rebrands every pre-login page with no code changes required.

Demo Mode

From Workspace Settings, the workspace owner can load a full set of realistic demo data into their own tenant for demonstrations, screenshots, or evaluation — a sample company ("Acme Technologies," green branding), five demo users, a pre-verified demo domain with those users tracked as employees, roughly a dozen credential findings across multiple sources and every status value, several weeks of scan history, and matching audit log activity.

Every row this creates is tagged internally as demo data and scoped to that tenant only. "Remove Demo Workspace" deletes exactly those rows and restores the tenant's real branding — it never touches real data,

and never affects any other tenant. See `SECURITY.md` for the safety guarantees and `DEPLOYMENT.md` for the required one-time migration if you're adding this to an existing deployment that already has data.

Public Marketing Pages

Beyond the authenticated dashboard, the application includes a set of public, unauthenticated pages intended for prospective buyers/end users landing on the deployment:

Route	Purpose
`/`	Landing page — hero section, a trust banner ("Supports Multiple Trusted Breach Intelligence Providers"), feature badges (Self Hosted, White Label Ready, Ready to Deploy, Multi-Tenant, Executive Reports, REST API, Webhooks, Multiple Breach Intelligence Providers), and a product walkthrough.
`/about`	What GhostLeaks is, why it was built self-hosted, and the white-label philosophy.
`/documentation`	A documentation hub linking to the real Markdown files shipped with the project (`README.md` , `DEPLOYMENT.md` , `ARCHITECTURE.md` , `API.md` , `SECURITY.md` , `CHANGELOG.md` , `FAQ.md`), served via a small whitelisted route (`/documentation/<key>`) rather than duplicated as separate content.
`/privacy`	Privacy policy describing the self-hosted data model — a template for the operator to adapt, not a substitute for their own legal review.
`/terms`	Terms & conditions covering acceptable use, third-party dependency notice, and liability — also a template.
`/contact`	Business/support contact, a note on the (private) source repository, and support response-time expectations.

All public pages share one footer (`templates/partials/footer.html` , included via `{% block footer %}`) linking About, Documentation, Privacy Policy, Terms & Conditions, and Contact. Authenticated pages (dashboard, settings, etc.) do not render this footer — the block is empty by default in `base.html` , so only pages that explicitly include the partial show it.

Buyer Benefits

- Source code ownership under a commercial license — see `LICENSE.md` for exact terms.
- No dependency on a single breach-data vendor — swap or combine providers via configuration.
- No dependency on the seller's infrastructure — bring your own database, API keys, and hosting.
- Multi-tenant and white-label from the start, rather than retrofitted.

Business Use Cases

- An MSP offers "breach monitoring" as a line item to existing security clients under its own brand.
- A cybersecurity agency runs GhostLeaks as an internal tool to monitor client domains it already manages.
- A SaaS company embeds breach monitoring as a feature of a broader product using the REST API.
- An internal security team monitors its own company domain and staff accounts without sending that data to a third-party SaaS vendor.

FAQ

See `FAQ.md` for common questions from buyers and self-hosters.

Documentation Index

Document	Contents
`LICENSE.md`	Commercial license terms
`DEPLOYMENT.md`	Deployment instructions (host and Docker)
`BUYER_GUIDE.md`	Guide for new buyers/operators
`API.md`	Full REST API and webhook reference
`SECURITY.md`	Security model, boundaries, and known limitations
`ARCHITECTURE.md`	Internal design decisions and rationale
`CHANGELOG.md`	Version history
`SUPPORT.md`	How to get support
`ROADMAP.md`	Planned and under-consideration work
`FAQ.md`	Frequently asked questions

-e

2. Feature Guide

Feature Guide

This document walks through every feature in GhostLeaks Enterprise Edition: what it does, why it exists, and how to use it — for you as the operator, and for the end users on your deployment. Pair this with `API.md` if you're integrating the REST API into your own product, and `BUYER_GUIDE.md` for how to position and sell this to your own customers.

Who This Is For — and What You Save

MSPs and cybersecurity agencies. You already sell security services — this gives you a breach-monitoring line item to add to every client contract without building one from scratch. A production-ready version of this (multi-tenant isolation, domain-ownership verification, confidence scoring, PDF reporting, white-labeling, an audit trail, and the security hardening documented in `SECURITY.md`) is realistically months of engineering time if built in-house, not a weekend project. You're buying that time back. Because it's multi-tenant, one deployment serves every client you have — you're not standing up infrastructure per client, so your hosting cost stays roughly flat as you add clients, while your billable line item scales per client.

Software agencies and SaaS companies. If you're building breach-checking into an existing product, the REST API (§13) is a working integration surface on day one — email check, password check, risk score, all key-authenticated — instead of you writing your own provider integration, confidence-scoring logic, and rate-limiting from scratch. The multi-provider architecture (§19) means adding a second breach-data source later is one new file, not a rearchitecture.

Internal security teams. The realistic comparison here isn't "build it yourself," it's the commercial breach-monitoring SaaS tools already on the market — most of which charge recurring per-seat or per-domain fees indefinitely. This is a one-time license instead of an ongoing subscription, and because it's self-hosted, your employees' email addresses and exposure data never leave your own infrastructure to sit in a third-party

vendor's database — which itself simplifies any internal data-handling or vendor-risk review you'd otherwise have to go through to approve a SaaS tool.

Across all three: you're also getting the parts that are easy to skip when building something like this quickly but expensive to bolt on later — tenant isolation enforced at the query level, domain-ownership verification before any domain-wide scan runs, API keys stored as hashes rather than plaintext, SSRF-guarded outbound webhooks, and self-service GDPR export/delete (§17–18) that removes a data-protection objection before a security-conscious buyer even has to ask about it.

---

1. Accounts, Workspaces, and Tenants

What it is: Every user belongs to exactly one *tenant* (a workspace — typically one client company, or your own organization if you're the only user). All data — findings, scans, domains, audit logs — is scoped to a tenant and can never leak across tenants.

Why it exists: This is what makes the product usable by an MSP or agency serving multiple clients from one deployment, or an internal security team that just wants a single isolated workspace. Tenant isolation is enforced by filtering every database query on `tenant_id` — see `ARCHITECTURE.md` for the underlying rule.

How to use it:

- The first person to register with no invite link becomes the `owner` of a brand-new workspace.
- To bring teammates in, share your registration link with `?tenant_slug=your-workspace-slug` appended (find your slug in Admin → Workspace Settings), or build an invite flow around it in your own frontend if you're white-labeling.
- Roles: `owner` (one per workspace, full control, cannot be demoted), `admin` (can manage domains, branding, and members), `member` (can run scans and see their own data).

---

2. Email Breach Check

What it is: Checks a single email address against your configured breach-intelligence provider(s) and records the result.

Why it exists: This is the core product loop — a user wants to know "has my email shown up in a known breach," and this answers that question with a status (`leaked` / `safe` / `check_failed`) and, if leaked, which source(s).

How to use it: Dashboard → email check box → enter address → submit. Result is saved to Scan History and, if leaked, creates or updates a row in Findings. A `check_failed` result (the provider didn't respond) is deliberately shown differently from `safe` — a failed check is never silently treated as "nothing found," because that would be a false negative masquerading as good news.

---

3. Password Breach Check

What it is: Checks whether a password has appeared in known breach dumps, using the Have I Been Pwned k-anonymity range API — only a 5-character SHA-1 prefix of the password ever leaves your server; the actual password is never transmitted or stored anywhere.

Why it exists: Complements the email check — an email can be "safe" while its password (reused elsewhere) is already compromised. This is a separate, valuable signal.

How to use it: Dashboard → password check box → submit. Only the result (`leaked: true/false` , and how many times it's been seen) is stored — never the password itself, not even masked; the stored `input_val` is literally `*` characters equal to the password's length, for length-pattern purposes only.

---

4. Bulk CSV Scan

What it is: Upload a CSV of email addresses (up to 100 per scan, up to 3 scans per day per user, 1 MB file size cap) and check all of them in one pass.

Why it exists: For onboarding a client's existing employee list, or checking a batch of addresses without one-by-one manual entry.

How to use it: Dashboard → Bulk Scan → upload a `.csv` with one email per line (quotes are stripped automatically). Results show per-email status and a leaked/safe/failed summary. The daily-3-scans cap and 1 MB size cap exist to stop a single upload from exhausting provider quota or server memory — increase the limits in `app.py` (`MAX_CSV_BYTES`, the `today_bulk >= 3` check) if your provider quota and infrastructure can support it.

---

5. Domain Monitoring

What it is: Ongoing, DNS-verified monitoring of an entire company domain — not just one person's email.

Why it exists: The realistic version of "monitor our whole company," done honestly: breach-intelligence providers index individual email addresses, not domain strings, so scanning the bare domain name rarely finds anything real. This feature verifies domain ownership first, then lets you add each employee's actual email so the scan checks real, indexed addresses.

How to use it (as an admin):

1. **Add the domain** — Dashboard → Domains → enter `example.com`. This creates an unverified `DomainMonitor` row with a random verification token.
2. **Verify ownership** — add a DNS TXT record at `_ghostleaks-verify.example.com` with the value `ghostleaks-verify=<token>` shown on the page, then click Verify. This step exists specifically so a tenant can't enumerate breach data for a domain it doesn't control.
3. **Add employee emails** — once verified, paste a list of emails (comma- or newline-separated, up to 200 per submission). Anything not ending in `@example.com` is silently skipped and reported back as skipped — this keeps a domain's tracked list scoped to that domain, on purpose.
4. **Scan** — Domains → Scan. If employee emails are tracked, every one of them gets checked individually and results roll up under the domain. If none are tracked yet, a best-effort fallback scans the bare domain string (rarely useful — the UI says so explicitly).

---

6. Findings

What it is: The persistent record of every discovered exposure — one row per (tenant, email, breach source) combination, deduplicated automatically. Each finding has a status you can move through a lifecycle: `active` → `rotated` / `historical` / `accepted_risk` / `resolved`.

Why it exists: A breach check is a point-in-time answer; Findings is what turns repeated checks into an ongoing security workload your team can actually track and close out — "we know about this, we changed the password, it's resolved" — rather than re-discovering the same exposure every scan.

How to use it: Findings page → filter by status. Click into a finding to change its status. Only the finding's owner or a tenant admin/owner can change status. Confidence score (0–100) increases with the number of independent providers that confirm the same exposure, and slightly with how long it's been publicly exposed — see `compute_confidence()` in `app.py` if you want to tune the weighting.

---

7. Exposure Timeline

What it is: A chronological feed of every “discovered” and “status changed” event across a tenant’s findings.

Why it exists: Gives a manager or auditor a single view of “what happened, in order” — useful for board-level security reporting, or a client-facing evidence trail of what was found and when it was remediated.

How to use it: Timeline page (nav) or `GET /api/timeline` if you’re building your own view on top of it.

---

8. Executive Risk Score

What it is: A single 0–100 score (and Low/Medium/High/Critical label) per user, computed from their scan history: the ratio of leaked-to-total scans, the number of distinct breach sources involved, and how recently the most recent leak occurred.

Why it exists: Non-technical stakeholders don’t want a list of breach names — they want “how worried should I be, on one number.” This exists to answer that in a client-facing dashboard or report without requiring the viewer to interpret raw findings.

How to use it: Shown on the Dashboard automatically. Also available as `GET /api/risk-score` (session-authenticated) or `GET /api/v1/risk-score` (API-key authenticated) if you want to surface it in your own UI. The exact weighting is intentionally simple (see `calculate_risk_score()`) — treat it as a directional indicator, not a certified risk metric, and say so to your own customers if you present it as such.

---

9. PDF Exposure Reports

What it is: A downloadable, tenant-branded PDF summarizing either a single email check or a full bulk-scan result.

Why it exists: End users and MSP clients frequently want something they can save, forward, or attach to a compliance file — a PDF is the lowest-friction “proof of check” artifact.

How to use it: After any email or bulk check, a “Download Report” action posts the result data to `/download-report`, which renders it server-side with ReportLab, using the tenant’s brand color, name, and logo. Every piece of user-controlled text in the PDF is XML-escaped before being placed into the ReportLab markup — this is a deliberate SSRF/injection guard (ReportLab’s markup will actually fetch an `<img src="...">` target if you don’t escape it) and should not be removed if you modify this code.

---

10. White-Label Branding

What it is: Per-tenant brand name, primary color, and logo, injected into every template a logged-in user sees.

Why it exists: This is the feature that makes the product resellable as “your own product” rather than “GhostLeaks with your logo bolted on” — an MSP’s client never sees the word “GhostLeaks” anywhere in their session.

How to use it: Admin (owner only) → Workspace Settings → set brand name, hex color, and logo URL. Logged-out pages (landing, login, register) use the deployment-wide default branding instead (`DEFAULT_BRAND_NAME` / `DEFAULT_BRAND_COLOR` / `DEFAULT_BRAND_LOGO` environment variables), since those pages aren’t tied to any one tenant yet.

---

11. Demo Mode

What it is: One click populates your *own* workspace with realistic, clearly-tagged sample data — users, a

domain, findings, scan history, and audit log entries spanning a fabricated four weeks — for demos, screenshots, and evaluation. One more click removes exactly that data and restores your real branding.

Why it exists: You need to demo this product to prospects without either showing an empty dashboard or using real client data. Every row this creates is tagged `is_demo=True` and scoped to your tenant — removal only ever touches those exact rows, in your tenant, never anything else.

How to use it: Admin (owner only) → Workspace Settings → Load Demo Data / Remove Demo Data. Refuses to load twice in a row (avoids duplicate data) and refuses to remove when nothing is loaded.

> **Note for an existing deployment upgrading into a version that already has real data:** Demo Mode added several `is_demo` columns and two `Tenant` columns. A fresh deploy gets these automatically from `db.create_all()`; an existing database needs them added manually first — see `DEPLOYMENT.md`.

---

12. Audit Log

What it is: An admin-visible, per-tenant record of who did what, when — logins, logouts, findings status changes, domain additions, branding changes, API key regeneration, demo data load/remove, account deletions, and more.

Why it exists: Distinct from Scan History (which records *what was scanned*): this records *administrative and state-changing actions*, which is what a client or compliance reviewer actually wants when they ask "who changed this."

How to use it: Admin → Audit Logs (owner/admin only), shows the most recent 200 entries for the tenant.

---

13. REST API (X-API-Key)

What it is: A small, deliberately scoped API surface — email check, password check, and risk score — authenticated with an `X-API-Key` header instead of a session cookie.

Why it exists: For embedding breach-checking into your own product rather than presenting the GhostLeaks UI directly — the primary integration path for software agencies and SaaS companies, per `BUYER_GUIDE.md`.

How to use it: Full endpoint reference, request/response shapes, and rate limits are in `API.md`. A key is auto-generated the first time a user visits Settings, and can be regenerated at any time (invalidating the old one immediately). Keys are stored as a SHA-256 hash, never in plaintext — the raw value is shown exactly once, at generation time.

---

14. Webhooks (Outbound)

What it is: When a breach is detected for a user (via manual check, scheduled alert, or API call), GhostLeaks can POST a JSON payload to a URL that user configured in Settings.

Why it exists: Lets a downstream system (Slack, a ticketing system, your own alerting pipeline) react to a breach the moment it's found, instead of polling.

How to use it: Settings → Webhook URL (must be `https://`). Every send re-validates the URL is not pointing at a private/internal/loopback address *immediately before sending*, not just at save time — this guards against DNS-rebinding, where a hostname resolves to a public IP when saved but a private one (e.g. a cloud metadata endpoint) later. Redirects are disabled on the outbound request for the same reason.

---

15. Scheduled Breach Alerts

What it is: A route (`/send-breach-alerts`) that, when triggered, checks every verified non-demo user's email against your breach providers and emails them if something new turned up since their last check.

Why it exists: This is what makes "we monitor you continuously" true rather than "we monitor you only when you happen to open the app." It's a route, not a background job — GhostLeaks does not run its own scheduler.

How to use it: You must trigger it yourself on a schedule — a cron job, your host's scheduled-task feature, or an external uptime/cron service hitting the URL. Authenticated with a shared secret in a custom header (`X-Alert-Secret` , matched against the `ALERT_SECRET` environment variable using a constant-time comparison) — not a query parameter, since query parameters leak into logs and browser history. See `DEPLOYMENT.md` §6 for a concrete cron example.

---

16. Billing (Stripe)

What it is: A one-time-payment upgrade flow using Stripe Checkout — a hosted payment page, not a custom card form — that marks a user `is_paid` once payment is confirmed.

Why it exists: The default gate for monetizing your own deployment directly (as opposed to charging customers outside the product entirely, e.g. as part of a services contract). Kept deliberately simple: one plan, one price, no subscription/metering logic, so it's easy to replace if your pricing model is more complex.

How to use it:

1. Create your own Stripe account and get a secret key from **Developers → API keys**.
2. Set `STRIPE_SECRET_KEY` in your environment.
3. Add a webhook endpoint in Stripe pointing at `https://<your-domain>/webhooks/stripe` , listening for `checkout.session.completed` , and set the resulting signing secret as `STRIPE_WEBHOOK_SECRET` .
4. Set `PLAN_AMOUNT_CENTS` , `PLAN_CURRENCY` , and `PLAN_PRICE_LABEL` to your own price — there is no hardcoded price or currency in the code.
5. Activation happens via the webhook, not a client-side callback — this is intentional; a client that never calls back (network drop, closed tab) doesn't leave a paid customer un-activated, and a forged client-side "success" call can't activate an account that didn't actually pay.

Order-ownership binding and payment-ID uniqueness (each Checkout session is tied to the user who created it, and a given Stripe payment can only ever activate one account) carry over from the original Razorpay implementation's security fixes — see `CHANGELOG.md` [1.0.1] and [1.0.2].

---

17. Self-Service Data Export (GDPR)

What it is: A logged-in user can download everything their account owns — profile fields, scan history, credential findings, audit-log entries — as a single JSON file, on demand.

Why it exists: A "right to access/portability" request is one of the first questions many buyers (especially UK/EU-facing ones) get asked, and being able to say "there's a button for that" removes an objection during a sale. It's genuinely self-service — no admin has to manually pull and send the data.

How to use it: Settings → "Export my data." Never includes the password hash, API key hash, or any other user's data.

---

18. Self-Service Account Deletion (GDPR)

What it is: A logged-in user can permanently delete their own account, after confirming their current

password.

Why it exists: The “right to erasure” counterpart to data export — same sales-objection reasoning.

How to use it: Settings → “Delete my account” → confirm password. Two situations are blocked outright, with a clear on-screen explanation, rather than silently working around them: - A workspace **owner** cannot self-delete while other members still exist in the tenant (would orphan the workspace — there is no ownership-transfer feature as of this writing). - A user who has **added a monitored domain** cannot self-delete until that domain is removed (the database foreign key from domain to the user who added it does not allow a dangling reference).

What actually happens on deletion: scan history (exclusively the user’s own) is hard-deleted. Credential findings and audit-log entries — shared organizational security data the rest of the tenant may still need — are kept, but the personal link to the deleted user is removed rather than the rows themselves.

---

19. Multi-Provider Breach Intelligence

What it is: The breach-checking backend supports more than one data provider at once (BreachDirectory, HIBP, LeakCheck, XposedOrNot as shipped), configured via `BREACH_PROVIDER` (a single name or comma-separated list).

Why it exists: A single provider’s coverage is incomplete, and its confidence ceiling is capped low on purpose (see Findings above) to reflect that honestly. Running more than one provider and letting findings get cross-confirmed is what earns a higher confidence score — and is a stronger sales pitch than “we use one API.”

How to use it: Set `BREACH_PROVIDER=breachdirectory,hibp` (etc.) and the matching API key environment variables for each. Adding a new provider entirely means writing one new `providers/*.py` module implementing the shared interface (see `providers/base_provider.py`) and adding one line to `_build_active_providers()` in `app.py` — nothing else in the file needs to change.

Where to go next

- **Deploying this for the first time:** `DEPLOYMENT.md`
- **Selling and positioning this to your own customers:** `BUYER_GUIDE.md`
- **Full REST API reference:** `API.md`
- **System design and the rules every contributor/maintainer should know:** `ARCHITECTURE.md`
- **What’s changed release over release:** `CHANGELOG.md` -e

3. System Architecture

Overview

This document explains how GhostLeaks is built and how it works internally — the request lifecycle, the data model, the breach-checking pipeline, and the security mechanisms underneath every feature. It’s written for whoever will run, extend, or evaluate the codebase: your own engineers, a technical buyer doing due diligence, or you.

For *what each feature does and why*, see `FEATURE_GUIDE.md`. For *how to deploy it*, see `DEPLOYMENT.md`. This document is about *how it’s built*.

---

1. Technology Stack

Layer	Choice	Why
Web framework	Flask 3.x	Small, explicit, no hidden magic — every route is a plain function you can read top to bottom.
ORM / database	Flask-SQLAlchemy 2.x, over PostgreSQL (production) or SQLite (local evaluation)	SQLAlchemy's engine abstraction means the same model code runs against either database with one connection-string change.
Auth	Flask-Login (session cookies) + a separate API-key path for the REST API	Two different consumers (a human in a browser, a machine calling the API) get two different, purpose-built auth mechanisms rather than one mechanism stretched to cover both.
Password hashing	Flask-Bcrypt	Industry-standard adaptive hashing; never reversible, never logged.
Payments	Stripe Checkout (hosted page) + webhooks	No card data ever touches this server — Stripe's hosted page handles that, and this app only ever sees a signed confirmation event.
Transactional email	Brevo (Sendinblue) API	Email verification, password reset, and breach-alert emails.
Breach intelligence	Pluggable provider modules (BreachDirectory, HIBP, LeakCheck, XposedOrNot)	See §5 — no single vendor is hardcoded into the application logic.
PDF generation	ReportLab	Server-side rendering of tenant-branded exposure reports.
Rate limiting	Flask-Limiter, in-memory by default, Redis-backed if `REDIS_URL` is set	See §7.
WSGI server	Gunicorn	Standard production entry point (`gunicorn app:app`).

The application is a single Flask app (`app.py`) — there is no microservice split, no message queue, no background worker process. Every request is handled synchronously, start to finish, by a Gunicorn worker. This is a deliberate simplicity choice: at the scale a single self-hosted deployment (or even a multi-tenant MSP deployment with dozens of client workspaces) actually runs at, a job queue adds operational complexity without a corresponding benefit. See §9 for where this stops being true.

---

2. Request Lifecycle

A typical browser request — say, a logged-in user running an email breach check — flows like this:

1. **Gunicorn** accepts the HTTP request and hands it to a Flask worker process.
2. **Flask-Login** resolves the session cookie to a `User` row (`load_user()`), making `current_user` available for the rest of the request. If there's no valid session, `current_user` is an anonymous user and any `@login_required` route redirects to `/login` .
3. **Flask-Limiter** checks the request against the route's configured rate limit (e.g. `"30 per minute"` on `/check-email`) before the view function runs at all. Over the limit → an immediate `429` with `Retry-After` headers, no application logic executes.
4. **CSRF protection** (Flask-WTF) validates the submitted token against the session for any state-changing `POST` request that isn't explicitly exempted (the two API-key-authenticated JSON endpoints are

exempted, since they don't use session cookies in the first place — see §6).
5. The **view function** runs: reads `request.form` / `request.get_json()`, does its work, and returns a response.
6. `@app.after_request` attaches baseline security headers (CSP, `X-Frame-Options`, HSTS, etc.) to every response before it goes back to the browser — this runs regardless of which route handled the request, so no route can accidentally ship without them.

For an API request (`X-API-Key` header instead of a session cookie), steps 2–4 are replaced by `require_api_key()`: it hashes the provided key, looks up the matching `User` by that hash, and attaches it to `request.api_user` — the rest of the request proceeds identically.

---

3. Data Model

Six tables, all scoped to a tenant except where noted:

- `Tenant` — one row per workspace/client company. Holds branding (`brand_name`, `primary_color`, `logo_url`), plan, and demo-mode state. This is the isolation boundary everything else hangs off of.
- `User` — belongs to exactly one tenant, has a `role` (`owner` / `admin` / `member`), a bcrypt password hash, an API key (stored as a SHA-256 hash — see §6), payment status, and webhook URL.
- `ScanHistory` — one row per breach check performed (email, password, or bulk), what it found, and when. This is *usage* data — "what was scanned."
- `CredentialFinding` — one row per (tenant, email, breach source) combination, deduplicated and upserted on every re-scan rather than duplicated. This is *state* data — "what's currently true about this exposure," carrying a status lifecycle (`active` → `rotated` / `historical` / `accepted_risk` / `resolved`) and a confidence score.
- `DomainMonitor` + `DomainEmployeeEmail` — a company domain a tenant wants monitored as a whole, gated behind DNS-based ownership verification (§4), with individual employee emails tracked underneath it.
- `AuditLog` — administrative and state-changing actions (logins, status changes, domain verification, branding changes, API key rotation, account deletion). Distinct from `ScanHistory`: this answers "who changed what," not "what was scanned."

Every table except `Tenant` itself carries a `tenant_id` foreign key, and every query in the codebase filters on it. This is enforced by code convention, not by a database-level policy (no Postgres row-level security) — see §9 for the implication of that.

---

4. Multi-Tenancy and Isolation

A tenant is created one of two ways: - **Register with no invite link** → a brand-new `Tenant` is created, and the registering user becomes its `owner`. - **Register with** `?tenant_slug=...` (an admin's invite link) → joins that existing tenant as a `member`.

From that point forward, every database query touching tenant-scoped data filters on `current_user.tenant_id` (or `request.api_user.tenant_id` for API calls). There is no code path anywhere in the application — UI or API — that returns data belonging to a different tenant. This is the single rule the whole multi-tenant model rests on, and it's why the product can safely serve multiple unrelated client companies from one deployment.

---

5. The Breach-Checking Pipeline

This is the core of the product, and it's built to be provider-agnostic on purpose.

Provider architecture: each breach-data source (BreachDirectory, HIBP, LeakCheck, XposedOrNot as shipped) is a small module in `providers/`, implementing one shared interface defined in `providers/base_provider.py`. Which provider(s) are active is controlled entirely by the `BREACH_PROVIDER`

environment variable — a single name, or a comma-separated list to run several at once. Adding a new data source later means writing one new provider module and adding one line to the factory function in `app.py`; nothing else in the application needs to change.

What actually happens on a check: 1. `aggregate_breach_sources(email)` calls every currently-active provider. 2. Each provider returns a list of source names it found the email in, or signals it couldn't respond (`check_failed`). 3. Results are tallied by source name across all providers — if two providers both report "LinkedIn," that's one finding with `confirmed_by = 2`, not two separate findings. 4. `compute_confidence()` scores the finding: 60 as a base for a single-source hit, +20 per additional confirming source (capped at +40), +5 if the exposure has been outstanding over a year. With only one provider configured, no finding can ever be cross-confirmed, so everything caps at 60–65 — that's the pipeline being honest about single-source confidence, not a bug. 5. The result is upserted into `CredentialFinding`, keyed on `(tenant_id, email, source_name)` — re-scanning the same email against the same source updates the existing row (`last_seen`, confidence) instead of creating a duplicate, so status history (§3) survives repeat scans.

A hard rule that runs through every one of these code paths: the actual leaked password or credential value is never read out of a provider's response, never stored, and never returned to a user — only the fact of exposure (which source, roughly when, how confident). Provider modules are written to not even parse that field out of the response. This is what makes the product a breach-*notification* tool rather than a credential-lookup tool, and it's treated as non-negotiable — see `SECURITY.md`.

Domain-wide monitoring (§ in `FEATURE_GUIDE.md`) runs the same pipeline once per tracked employee email, rather than trying to check a bare domain string — breach providers index individual addresses, not domains, so that's what actually finds real exposures. It's gated behind a live DNS TXT-record check before any domain-wide scan is allowed to run, so a tenant can't enumerate exposure data for a domain it doesn't control.

---

6. Authentication and API Security

Session-based (browser) auth: standard Flask-Login session cookies, hardened with `HttpOnly`, `SameSite=Lax`, and `Secure` (over HTTPS). CSRF tokens are required on every state-changing form submission.

API-key auth: a key is generated the first time a user visits Settings (`gl_` prefix + 48 hex characters), and can be regenerated at any time, immediately invalidating the old one. Keys are stored as a SHA-256 hash — the raw value is shown exactly once, at generation time, and is never recoverable from the database afterward. The two REST endpoints that use this (`/api/v1/check-email`, `/api/v1/check-password`) are explicitly exempted from CSRF checks, since CSRF protection exists to defend session-cookie auth specifically — a request authenticated by a header the browser doesn't attach automatically isn't vulnerable to the cross-site form-submission attack CSRF tokens defend against.

Outbound webhooks: before every send (not just when the URL is first saved), the destination hostname is resolved and checked against private/loopback/link-local/reserved IP ranges, and the outbound request has redirects disabled. Both exist to close a specific gap: a hostname can resolve to a public IP when saved and a private one later (DNS rebinding), and `requests` follows redirects by default, which would otherwise let an allowed `https://` host 30x to an internal address.

Payment webhook: Stripe webhook payloads are verified against their signature (using `STRIPE_WEBHOOK_SECRET`) before being trusted — an unsigned or incorrectly-signed request is rejected outright, so activation can't be triggered by anyone who doesn't hold Stripe's actual signing secret.

Scheduled-job auth: the breach-alert cron endpoint (`/send-breach-alerts`) is protected by a shared secret compared using `hmac.compare_digest` (constant-time, to avoid timing attacks) and passed via a custom header rather than a query parameter — query parameters end up in server logs and browser history, a header does not.

---

7. Rate Limiting

Flask-Limiter enforces per-route limits (e.g. 10 registrations/hour, 30 checks/minute, 3 bulk scans/day) to prevent abuse and control third-party provider API spend. By default this state lives in-memory inside each Gunicorn worker process — which means the *effective* limit is roughly the configured value multiplied by however many workers are running, and resets whenever a worker restarts. Setting `REDIS_URL` switches to a shared Redis-backed store, giving accurate limits across every worker and every restart. For a single-worker deployment this distinction doesn't matter; for anything scaled beyond one worker, Redis is the correct configuration, not an optional upgrade.

---

8. Payment Architecture

Stripe Checkout is used as a **hosted page**, not a custom card form — the browser is redirected to a page Stripe controls, and this application never sees, handles, or stores any card data. Two things happen on the way there and back:

- `/create-checkout-session` creates a Stripe Checkout Session tied to `current_user`, recording which order belongs to which user before redirecting.
- `/webhooks/stripe` receives Stripe's `checkout.session.completed` event, verifies its signature, and only then marks the user `is_paid`.

Activation is driven entirely by the webhook, not by a client-side "payment succeeded" callback — a webhook still fires even if the customer's browser closes or their network drops right after paying, and a webhook can't be forged into activating an account that didn't actually pay, the way a client-side callback could be. Price and currency are read from environment variables (`PLAN_AMOUNT_CENTS`, `PLAN_CURRENCY`, `PLAN_PRICE_LABEL`) — nothing is hardcoded, so a deployment can price itself in any Stripe-supported currency without a code change.

---

9. Known Architectural Boundaries

Every system has edges. These are the ones worth knowing before you extend or scale this one:

- **No database migration tool.** Schema changes rely on `db.create_all()`, which creates missing *tables* but never adds a missing *column* to a table that already has data in it. A fresh deployment is unaffected; an existing production database needs the exact `ALTER TABLE` statement run manually for each schema change (documented per-change in `DEPLOYMENT.md`). Adopting a migration tool (e.g. Flask-Migrate/Alembic) is the natural next step if the schema will keep evolving.
- **No database-level tenant isolation.** Isolation is enforced by every query filtering on `tenant_id` in application code, not by a Postgres row-level security policy. For most deployments this is sufficient; if you're serving mutually distrustful tenants at a scale where a single missed filter would be catastrophic, RLS is a reasonable second line of defense to add.
- **No background job queue.** Bulk CSV scans and the scheduled breach-alert job run synchronously, inside the request or cron-triggered request itself. This is fine at the scale currently tested; a very large bulk upload or a very large user base on the scheduled job could approach a request timeout before a queue-based redesign becomes necessary.
- **Rate-limit storage defaults to in-memory** (see §7) — correct for a single worker, needs `REDIS_URL` beyond that.
- **No automated test suite.** Changes are verified by static syntax/import checks and manual testing, not a CI test suite exercising application behavior.

None of these are defects — they're the tradeoffs of a codebase sized correctly for what it actually needs to do today, documented honestly so whoever operates or extends it knows exactly where the edges are. -e

4. REST API Reference

API Reference

This document covers GhostLeaks Enterprise Edition's REST API (for external integrations), its webhook payload format, and the internal session-based endpoints used by the web dashboard itself.

Authentication

The public REST API is authenticated with a per-user API key.

- Every user has one API key, generated automatically on registration and viewable/regenerable from **Settings**.
- Send it as the `X-API-Key` request header on every call.
- Regenerating a key immediately invalidates the previous one — any integration using the old key will start receiving `401 Invalid API key` until updated.
- API access requires the account to be on a paid plan; unpaid accounts receive `402 UPGRADE_REQUIRED`.

```
curl -X POST https://your-deployment.example.com/api/v1/check-email \
  -H "X-API-Key: gl_your_api_key_here" \
  -H "Content-Type: application/json" \
  -d '{"email":"someone@example.com"}'
```

CSRF protection does not apply to these endpoints — they are exempted from it because they authenticate via header, not session cookie.

Rate Limits

Endpoint	Limit
`POST /api/v1/check-email`	60 requests / minute
`POST /api/v1/check-password`	60 requests / minute
`GET /api/v1/risk-score`	60 requests / minute
All other routes (default)	500 / day, 100 / hour per client IP

> Rate limiting is implemented with an in-memory store per application process. If you run multiple Gunicorn workers or multiple app instances, each maintains its own independent counters — see `SECURITY.md` for the implication. For accurate shared limits across workers/instances, back the limiter with Redis instead of the default in-memory store.

Endpoints

`POST /api/v1/check-email`

Checks a single email address against the currently configured breach-data provider(s).

Request body

```
{
  "email": "someone@example.com"
}
```

Response — `200 OK`

```
{
  "email": "someone@example.com",
  "leaked": true,
  "breach_count": 2,
  "breaches": [
    {
      "name": "ExampleForum2019",
      "confidence_score": 65,
      "status": "active",
      "finding_id": 123
    }
  ]
}
```

`status` is one of: `active`, `rotated`, `historical`, `accepted_risk`, `resolved` — see `ARCHITECTURE.md` for the credential-status lifecycle. Note that no password, hash, or other credential value is ever included in the response — only source name, confidence score, and status.

Error responses

Status	Body	Meaning
`400`	`{"error": "email field is required"}`	Missing `email` in request body.
`400`	`{"error": "Breach checking is not configured on this server. Contact the administrator."}`	No breach provider API key is configured server-side.
`401`	`{"error": "Missing X-API-Key header"}` / `{"error": "Invalid API key"}`	Authentication failure.
`402`	`{"error": "UPGRADE_REQUIRED"}`	Account is not on a paid plan.
`502`	`{"error": "CHECK_FAILED", "message": "..."}`	The breach-data provider did not respond. This is explicitly **not** the same as a confirmed-safe result — treat it as “unknown,” not “clean.”

If any breach is found, a webhook is fired for the account (see Webhooks below) and a scan-history entry is recorded.

`POST /api/v1/check-password`

Checks a password using k-Anonymity (Have I Been Pwned’s range API). Only a 5-character hash prefix of the password is ever transmitted externally — see `SECURITY.md`.

Request body

```
{
  "password": "the-password-to-check"
}
```

Response — `200 OK`

```
{
  "leaked": true,
  "times_seen": 41235
}
```

Error responses

Status	Body	Meaning

`400`	`{"error": "password field is required"}`	Missing `password` in request body.
`401`	see above	Authentication failure.
`402`	see above	Account is not on a paid plan.

`GET /api/v1/risk-score`

Returns the authenticated user's current Executive Risk Score.

Response — `200 OK`

```
{
  "email": "someone@example.com",
  "risk_score": 68,
  "risk_level": "Medium",
  "calculated_at": "2026-07-26T10:15:00.000000"
}
```

`risk_level` is one of: `Low`, `Medium`, `High`, `Critical`, `Unknown`.

Webhooks

Each user can configure one webhook URL from **Settings**. When a new breach is detected for that user — via `POST /api/v1/check-email` or the scheduled alert job — GhostLeaks sends an HTTP `POST` with a JSON body to that URL.

Requirements for the webhook URL:

- Must use `https://`.
- Must not resolve to a private, loopback, link-local, reserved, or multicast IP address (blocks internal network / cloud metadata targets). This is re-checked immediately before every send, not just when the URL is saved.
- Redirects are not followed on delivery.

Payload shape

```
{
  "event": "breach_detected",
  "user_email": "someone@example.com",
  "timestamp": "2026-07-26T10:15:00.000000",
  "data": {
    "email": "someone@example.com",
    "breach_count": 2,
    "sources": ["ExampleForum2019", "ExampleShop2021"]
  }
}
```

Delivery is best-effort: failures are logged server-side and are not retried automatically.

Internal Endpoints (session-authenticated, not part of the public integration API)

These power the web dashboard itself. They require an active logged-in session (not an API key) and are not intended for third-party integration — they exist here for completeness, not as a stable external contract.

Method	Endpoint	Purpose

`GET`	`/api/risk-score`	Returns `{risk_score, risk_level}` for the logged-in user. Session-authenticated equivalent of the public risk-score endpoint.
`GET`	`/api/timeline`	Returns exposure and status-change events for the logged-in user's tenant (all tenant findings if admin, own findings only otherwise), used to render `/timeline`.
`POST`	`/findings/<id>/status`	Updates a finding's status (`active` / `rotated` / `historical` / `accepted_risk` / `resolved`). Used by the Findings page.

Versioning

The public API is versioned under `/api/v1/`. -e

5. Security

Security Policy

This document describes GhostLeaks Enterprise Edition's security model, the protections already built into the codebase, known limitations, and how to report a vulnerability.

> This document describes the security posture of the source code as licensed. It is not a certification, audit report, or compliance attestation. Licensee is responsible for its own security review, penetration testing, and compliance obligations before operating the Software in production — see `LICENSE.md` Section 6.

Reporting a Vulnerability

Ritesh — ritdmdkkd@gmail.com

Please include:

- A description of the vulnerability and its potential impact.
- Steps to reproduce, including any affected endpoint(s).
- Whether the issue has been disclosed elsewhere.

Do not open a public issue for security vulnerabilities if this repository or any fork of it is ever made accessible to others — report privately instead. Note that redistributing or publishing this source code publicly is itself a violation of the commercial license.

Built-in Security Measures

The following protections exist in the codebase as delivered. This list reflects what is actually implemented, not aspirational goals.

Credential handling

- **Passwords are checked via k-Anonymity.** Only a 5-character SHA-1 prefix of the password is ever sent to the password-check provider (Have I Been Pwned's range API); the full password never leaves the point where it is hashed.

- **Breach provider responses are filtered before use.** Every provider implementation returns only a source name — never a password, hash, or other credential value from the underlying breach data. This is enforced at the provider layer (`providers/`), not only in the UI, so it cannot be bypassed by a template change.
- **Passwords are hashed with bcrypt** (`Flask-Bcrypt`) before storage; plaintext passwords are never persisted.

Domain monitoring

- Domain-wide monitoring requires **DNS TXT ownership verification** before any scan can run against a domain. A tenant cannot monitor a domain it hasn't proven control of.

Webhooks (SSRF protections)

Webhook URLs are validated to reduce server-side request forgery risk:

- Only `https://` URLs are accepted.
- The resolved IP address(es) of the hostname are checked and rejected if private, loopback, link-local, reserved, or multicast (blocking access to internal networks and cloud metadata endpoints).
- This check is re-run **immediately before every webhook send**, not only when the URL is saved — this specifically defends against DNS rebinding, where a hostname resolves to a public address at save time and an internal address later.
- Redirects are disabled on the outbound webhook request (`allow_redirects=False`), so a webhook cannot be saved pointing at an allowed host and then redirect to an internal one at send time.

Session and request security

- Session cookies are set `HttpOnly` and `SameSite=Lax` .
- `SESSION_COOKIE_SECURE` is enabled automatically unless `FLASK_ENV=development` is set.
- CSRF protection (`Flask-WTF`) is enabled on session-authenticated forms. The three `/api/v1/*` REST endpoints are explicitly exempted from CSRF, since they authenticate via the `X-API-Key` header rather than a session cookie.
- Request body size is capped (`MAX_CONTENT_LENGTH` , 2 MB) to limit abuse via oversized uploads.
- Rate limiting (`Flask-Limiter`) is applied per-route (see `API.md` for the current per-endpoint limits) with a default fallback limit of 500/day and 100/hour per client IP.
- **Security headers** (v1.0.0) are set on every response via an `after_request` hook: `X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Referrer-Policy: strict-origin-when-cross-origin` , `Permissions-Policy` , and a `Content-Security-Policy` . `Strict-Transport-Security` is added only when the request is already HTTPS, so it's never sent on a plain-HTTP local/dev connection. The CSP allows `'unsafe-inline'` for scripts and styles because the UI's existing inline `<script>` blocks and `onclick` handlers depend on it — tightening this to a nonce-based CSP would require reworking every template's JavaScript, which was deliberately out of scope for the v1.0.0 launch (see `ROADMAP.md`) to avoid regression risk.
- The scheduled-alert endpoint's shared-secret check (v1.0.0) uses `hmac.compare_digest()` instead of `!=` , avoiding a timing side-channel on the comparison.
- `/health` (v1.0.0) is an unauthenticated liveness/readiness endpoint for load balancers and uptime monitors — it checks the app is responding and the database is reachable, returning `503` if the database check fails. It intentionally reveals no information beyond up/down status.
- **Payment ownership verification** (v1.0.0): `/verify-payment` previously verified the Razorpay signature (proving `{order_id, payment_id, signature}` are mutually consistent) but never confirmed the order belonged to the account trying to redeem it. A valid triple obtained any way — including a leaked or shared one — could activate a different account. Orders are now bound to the user who created them (`User.pending_order_id` , set in `/create-order`), checked before activation, and `razorpay_payment_id` is a unique database column so the same payment can never activate more than one account.
- **Rate-limiter storage is now configurable** (v1.0.0): set `REDIS_URL` to use a shared Redis backend instead of the default in-memory store, so limits are accurate across multiple Gunicorn workers or app instances rather than effectively multiplied by worker count. Unset by default — existing deployments are unaffected until this is explicitly configured.

- **API keys are hashed, not stored in plaintext** (v1.0.0): new and regenerated API keys are stored as a SHA-256 hash; the raw value is shown once at generation time and cannot be retrieved from the database afterward. Keys issued before this change continue to work via a backward-compatible plaintext lookup path — no existing integration was broken by this change, but regenerating (recommended) upgrades that account to the newer storage.

Data isolation

- Every tenant-scoped table (users, domains, findings, scan history) is queried with an explicit `tenant_id` filter. There is no code path that returns another tenant's data by default.
- Database access is entirely through SQLAlchemy's ORM query interface — there is no raw, string-concatenated SQL in the application.

Demo Mode

- Restricted to the `owner` role, and every query it runs (both loading and removing demo data) is scoped to `current_user.tenant` — it cannot affect any other tenant on the deployment.
- Every row it creates is tagged `is_demo = True`. Removal deletes only rows with that tag for that tenant; no other data, demo or real, is ever touched.
- Demo user accounts (`*@acme-demo.com`) are created with a random, never-disclosed password hash — they cannot be logged into. They exist only to populate lists (Members, Findings, Audit Log), not as usable accounts.
- The tenant's real branding (name, color, logo) is backed up before being overwritten with demo branding, and restored exactly on removal — never reset to a hardcoded default.
- Loading demo data twice is blocked by a `demo_active` flag on the tenant, rather than relying on the button being disabled client-side (which a direct POST request could bypass).

Audit trail

- Account and workspace-level actions (registration, status changes, domain actions, branding changes, API key regeneration) are recorded in an admin-visible audit log, separate from per-user scan history.

Known Limitations

These are honest, current gaps — not omissions. Review them before relying on this Software for a security-sensitive deployment.

1. **Role-based access control.** `owner`, `admin`, and `member` roles are all reachable today. The workspace owner can promote a member to `admin` or demote an admin back to `member` from Workspace Settings (`/admin/tenant-settings`), which requires the `owner` role. An owner cannot change their own role through this control, and a member cannot self-promote — only the owner can change another member's role, and every change is recorded in the audit log.
2. **Rate limiting is in-memory, per-process.** `Flask-Limiter` is configured with `storage_uri="memory://"`. Gunicorn typically runs multiple worker processes (`2 * CPU + 1` by default, per `gunicorn_config.py`), and each worker maintains its own independent rate-limit counters. This means the effective rate limit for a given client is roughly `(configured limit) × (number of workers)`, not the configured value exactly, and limits reset if a worker restarts. For accurate, shared rate limiting across workers or multiple app instances, point `Flask-Limiter` at a shared backend (e.g. Redis) instead.
3. **Domain-wide scanning without employee emails is a weak fallback.** Once a domain is DNS-verified, scanning it without first adding employee emails falls back to searching the bare domain string against the configured provider(s) — which rarely returns meaningful results, since breach providers index individual emails rather than domain names. This is documented in-product; don't present a domain scan as comprehensive coverage unless employee emails have been added.
4. **Confidence scoring is only as strong as the number of active providers.** With a single provider configured, no finding can be cross-confirmed, so confidence scores are honestly capped lower than they would be with multiple providers active.
5. **No built-in background job scheduler.** The scheduled breach-alert endpoint (`GET /send-breach-`

`alerts`) must be triggered by an external scheduler (host cron, a scheduled job service, etc.) — see `DEPLOYMENT.md`.

6. **No dedicated Web Application Firewall, intrusion detection, or anomaly-detection layer** is included. These are deployment-level concerns Licensee should address (e.g. a WAF or reverse proxy in front of the application) if required for its environment.

Dependency and Third-Party Risk

GhostLeaks depends on third-party services (breach-intelligence providers, payment processing, transactional email) and open-source Python packages (see `requirements.txt`). Licensee is responsible for:

- Keeping dependencies patched and up to date.
- Monitoring for vulnerabilities disclosed in dependencies (e.g. via `pip-audit`, Dependabot, or similar tooling — not currently configured in this project).
- Reviewing the terms, data handling practices, and security posture of any third-party provider it connects (see `LICENSE.md` Section 4).

-e

6. Deployment Guide

Deployment Guide

Two supported paths: **Docker Compose** (bundled PostgreSQL, recommended for most buyers) or a **manual host deployment** (your own Python environment and database). Both are covered below.

---

0. First hour — plain-language checklist

Do these in order, regardless of which deployment path you choose:

1. **Get your accounts ready** (10–15 min): a RapidAPI account with a BreachDirectory subscription (or an account with any other supported provider — see `README.md`), a Brevo account for sending email, and — if not using the bundled Docker database — a PostgreSQL database (a managed provider such as Render or Supabase both work).
2. **Fill in your environment variables.** Copy `.env.example` to `.env` and fill in real values. At minimum: `SECRET_KEY`, `ALERT_SECRET`, at least one breach-provider key (`RAPIDAPI_KEY` by default), `BREVO_API_KEY`, `MAIL_USERNAME` (an email address you actually control — the app will refuse to send email rather than silently use a placeholder address), and `APP_PUBLIC_URL` once you have a real domain.
3. **Start the app and load it once** — this triggers `db.create_all()`, which creates every required table automatically.
4. **Register your own account** through `/register` — you become the `owner` of a new workspace automatically. Check your email and click the verification link.
5. **Set your branding** at `/admin/tenant-settings` — your company name, primary color, and logo. This is what your customers or employees see instead of default GhostLeaks branding.
6. **If monitoring a company domain:** add it under the Domains tab, publish the DNS TXT record it shows you, click “Check TXT” once it’s propagated (a few minutes to a couple of hours depending on your DNS provider), then add the domain’s employee emails so scans check something real — see `README.md`.
7. **Wire up scheduled monitoring** — point an external scheduler at `GET /send-breach-alerts` with header `X-Alert-Secret: <your ALERT_SECRET>`, once every 24 hours. See §6.
8. **Invite your team** — share the invite link on the Workspace Settings page; anyone who registers through it joins your workspace instead of creating a new one.

If something isn't working, check the application logs first — most first-day issues are a missing or incorrect environment variable, and the app logs those explicitly rather than failing silently.

---

1. Option A — Docker Compose (recommended)

The project includes a `Dockerfile`, `docker-compose.yml`, and `.env.example` that together run the app alongside a bundled PostgreSQL container.

```
cp .env.example .env
# edit .env — fill in SECRET_KEY, ALERT_SECRET, POSTGRES_PASSWORD, provider keys, etc.
docker compose up -d
```

What this does:

- Builds the app image from the included `Dockerfile`.
- Starts a `postgres:16-alpine` container using `POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB` from `.env`.
- Waits for the database to report healthy (`pg_isready`) before starting the app — `docker-entrypoint.sh` also waits for Postgres to accept connections before running Gunicorn.
- Builds `DATABASE_URL` automatically from the `POSTGRES_*` values, so you do not need to construct a connection string by hand. If you'd rather point at an external database instead of the bundled container, uncomment and set `DATABASE_URL` directly in `.env` — that value takes precedence.
- Exposes the app on `${APP_PORT:-5000}` (default `5000`) on the host.

To stop: `docker compose down` (add `-v` to also remove the database volume — this deletes your data, so only do this intentionally).

Gunicorn worker/thread/timeout tuning is read from `GUNICORN_WORKERS`, `GUNICORN_THREADS`, `GUNICORN_TIMEOUT`, and `GUNICORN_LOG_LEVEL` in `.env` — see `gunicorn_config.py` for the defaults used when these are left unset.

The container runs as a non-root user (`appuser`) as of v1.0.1 — no action needed on your part, this is just how the image is built now.

Optional: shared rate limiting with Redis. By default, rate limits are tracked in memory per Gunicorn worker, which means the effective limit is roughly the configured value multiplied by your worker count (see `SECURITY.md`). If you're running multiple workers or multiple app instances and want accurate shared limits, bring up the bundled Redis service and point the app at it:

```
docker compose --profile with-redis up -d
```

then set `REDIS_URL=redis://redis:6379/0` in `.env` and restart the `web` service. A plain `docker compose up -d` (without `--profile with-redis`) never starts Redis and behaves exactly as before — this is entirely opt-in.

---

2. Option B — Manual host deployment

```
pip install -r requirements.txt
cp .env.example .env   # fill in real values, including a real DATABASE_URL
python3 app.py         # local dev
# or, in production:
gunicorn -c gunicorn_config.py app:app
```

`db.create_all()` runs automatically on startup and creates any tables that don't already exist. This is **additive-only** — it will not alter existing tables. If you are upgrading an existing production deployment rather than starting fresh, see §4 before deploying.

If you're using SQLite (leave `DATABASE_URL` unset entirely) for evaluation, be aware that SQLite is a single file

on local disk — it will not survive a container restart on most hosting platforms unless you mount a persistent volume for it, and it is not a recommended production configuration. Use PostgreSQL for anything beyond local evaluation.

Pin your Python version. This app is built and tested against Python 3.11 (the Dockerfile uses `python:3.11-slim`). If you're deploying to a platform with a native Python buildpack instead of Docker (Render, Railway, Heroku-style hosts), that platform will default to whatever its *latest* Python is unless you tell it otherwise — which can be a version this app has never been tested against, and dependency wheels (especially `psycopg` / `setuptools`) may not have compatible builds for it yet, causing install or startup failures that have nothing to do with your configuration. Add a `runtime.txt` file to the project root containing:

```
python-3.11.9
```

This is the single most common cause of "it worked locally but fails on my host" for buyers deploying without Docker — set this before anything else if you hit an install or startup error on a native-buildpack host.

If your database sits behind a connection pooler (e.g. Supabase's pooled connection string, PgBouncer in transaction mode): the app connects using `psycopg` (v3), which uses server-side prepared statements by default. Transaction-mode poolers can reuse the same underlying connection across unrelated sessions, which can produce a `DuplicatePreparedStatement` error under load. If you see that error, either use your provider's **session-mode / direct** connection string instead of the pooled one, or disable server-side prepares by adding this right after `app.config['SQLALCHEMY_DATABASE_URI'] = database_url` in `app.py`:

```
        if database_url.startswith('postgresql+psycopg://'):
            app.config['SQLALCHEMY_ENGINE_OPTIONS'] = {
                'connect_args': {'prepare_threshold': None}
            }
```

---

3. Environment Variables

See `.env.example` for the complete list with inline explanations. Summary:

Variable	Purpose
`SECRET_KEY`	Flask session/token signing. Required.
`ALERT_SECRET`	Protects the scheduled breach-alert endpoint. Required.
`DATABASE_URL`	Leave unset/commented when using Docker Compose (built automatically); set directly for an external database or manual deployment.
`POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB`	Used by Docker Compose to configure the bundled Postgres container.
`BREACH_PROVIDER`	Which breach-data provider(s) are active.
`RAPIDAPI_KEY` / `HIBP_API_KEY` / `LEAKCHECK_API_KEY` / `XPOSEDORNOT_API_KEY`	Provider API keys — only the ones for your selected provider(s) need real values.
`RAZORPAY_KEY_ID` / `RAZORPAY_KEY_SECRET`	Payment processing.
`BREVO_API_KEY` / `MAIL_USERNAME`	Transactional email.
`APP_PUBLIC_URL`	Your deployment's public URL, used in emails and reports.
`APP_PORT` / `GUNICORN_*`	Optional Docker/Gunicorn tuning.

`RAPIDAPI_KEY` (and any other provider key you configure) is shared server-wide across every tenant on this deployment, not per-tenant. If you're reselling to multiple client companies from one deployment, monitor your provider's quota — many small tenants share the same quota pool.

---

4. Upgrading an existing deployment

If you are upgrading from a version of this codebase that predates multi-tenancy (no `Tenant`, `DomainMonitor`, `CredentialFinding`, or `AuditLog` tables, and no `tenant_id` column on `User` / `ScanHistory`), `db.create_all()` will create the new tables but will **not** add a `tenant_id` column to your existing `user` / `scan_history` tables or backfill it. Running it as-is against such a database will fail or leave rows inconsistent.

Recommended path:

1. Take a full database backup first.
2. Manually create one `Tenant` row representing your existing single-company deployment.
3. Add a nullable `tenant_id` column to `user` and `scan_history`, backfill every existing row with that tenant's ID, then tighten the column to `NOT NULL` once backfilled.
4. Deploy normally — `db.create_all()` will create the remaining new tables.

This isn't scripted here because it depends on your existing data shape. Treat it as a one-time manual migration; adopting Alembic before this point is a reasonable investment if you don't already have a migration tool in place — see `ARCHITECTURE.md` for the current state of schema management (`db.create_all()` only, no migration framework).

Required migration: Demo Mode columns

Demo Mode (see §9) added new columns to six existing tables. `db.create_all()` only creates missing **tables**, not missing **columns** on tables that already exist — if you're upgrading a deployment that already has data, run this once against your database before Demo Mode will work (a fresh, brand-new deployment doesn't need this — `db.create_all()` already includes these columns for tables it creates from scratch):

```
ALTER TABLE tenant ADD COLUMN demo_active BOOLEAN DEFAULT FALSE;
ALTER TABLE tenant ADD COLUMN demo_branding_backup TEXT;
ALTER TABLE "user" ADD COLUMN is_demo BOOLEAN DEFAULT FALSE;
ALTER TABLE scan_history ADD COLUMN is_demo BOOLEAN DEFAULT FALSE;
ALTER TABLE domain_monitor ADD COLUMN is_demo BOOLEAN DEFAULT FALSE;
ALTER TABLE domain_employee_email ADD COLUMN is_demo BOOLEAN DEFAULT FALSE;
ALTER TABLE credential_finding ADD COLUMN is_demo BOOLEAN DEFAULT FALSE;
ALTER TABLE audit_log ADD COLUMN is_demo BOOLEAN DEFAULT FALSE;
```

Back up your database before running this, as with any schema change. `"user"` is quoted because `user` is a reserved word in PostgreSQL.

Required migration: API key hashing column

A separate fix stores API keys as a SHA-256 hash (`api_key_hash`) rather than relying solely on the plaintext `api_key` column. A fresh deployment already gets this column from `db.create_all()`; an existing deployment with data needs it added manually before `/settings` or `X-API-Key` auth will work correctly:

```
ALTER TABLE "user" ADD COLUMN api_key_hash VARCHAR(64) UNIQUE;
```

Required migration: Stripe payment column

The payment provider was switched from Razorpay to Stripe. A fresh deployment gets the `stripe_payment_id` column automatically from `db.create_all()`; an existing deployment (upgrading from a Razorpay-based version) needs it added manually, and the old Razorpay column dropped:

```
ALTER TABLE "user" ADD COLUMN stripe_payment_id VARCHAR(200) UNIQUE;
ALTER TABLE "user" DROP COLUMN IF EXISTS razorpay_payment_id;
```

Required migration: Stripe payment column

Payments moved from Razorpay to Stripe. A fresh deployment already gets the right column from `db.create_all()`. An **existing deployment that had the old `razorpay_payment_id` / `pending_order_id`**

columns needs this run manually — old paid-user data is preserved, it's just renamed:

```
ALTER TABLE "user" ADD COLUMN stripe_payment_id VARCHAR(200) UNIQUE;
ALTER TABLE "user" DROP COLUMN pending_order_id;
-- Optional cleanup once you've confirmed nothing else reads the old column:
-- ALTER TABLE "user" DROP COLUMN razorpay_payment_id;
```

You'll also need to configure a Stripe webhook. In your Stripe Dashboard → Developers → Webhooks, add an endpoint at `https://your-domain.example.com/webhooks/stripe` subscribed to the `checkout.session.completed` event, then copy the signing secret it gives you into `STRIPE_WEBHOOK_SECRET`. Without this, payments go through on Stripe's side but the account never gets marked paid — the webhook is what actually activates it, on purpose (see the comment on `payment_success()` in `app.py` for why the redirect page alone must never do this).

---

5. Domain Monitoring — DNS Setup

When a tenant admin adds a domain, they're shown a TXT record to publish:

```
Host:  _ghostleaks-verify.<their-domain>
Type:  TXT
Value: ghostleaks-verify=<random-token-shown-in-dashboard>
```

Most DNS providers propagate TXT records within a few minutes, but allow up to 24–48 hours before treating "not verified yet" as a real failure. The verify action re-checks live DNS on demand — there's no hardcoded waiting period.

Once verified, add the domain's employee emails before scanning — see `README.md` for why this matters for scan quality.

---

6. Scheduled Breach Alerts

`GET /send-breach-alerts` with header `X-Alert-Secret: <ALERT_SECRET>` is meant to be called by an external scheduler once a day — a host's cron/scheduled-job feature, a GitHub Actions scheduled workflow, or an external cron service. The application does not run this on its own; nothing inside the app process triggers it automatically.

It iterates every paid, email-verified user across every tenant, checks their email, and emails them only about *new* sources found since their last check. The secret is passed as a header rather than a query parameter so it doesn't end up in server access logs, browser history, or a proxy's request logs.

---

7. Multi-Tenant / White-Label Operational Notes

- Every new tenant is created with a default plan and default branding; update via `/admin/tenant-settings` (owner-only).
- The invite link on that page (`/register?tenant_slug=<slug>`) is what you hand to a client company so their team joins their own isolated tenant.
- Provider API keys are server-wide, not per-tenant, in the current version — see §3 above regarding quota sharing.

---

8. Backups

`[Insert your backup strategy here — e.g. your PostgreSQL provider's automated backup/point-in-time-`

`recovery feature, and how often you test a restore.]` This is deployment-specific and depends on where you host the database; the application itself does not implement its own backup mechanism.

---

9. Demo Mode

Workspace Settings (`/admin/tenant-settings` , owner-only) includes a Demo Mode card with two actions:

- **Load Demo Workspace** — populates the current tenant with a demo company (“Acme Technologies,” green branding, a placeholder logo), five demo users, a pre-verified demo domain (`acme-demo.com`) with those users tracked as employee emails, roughly a dozen credential findings spread across multiple sources and every status value, several weeks of scan history, and a matching spread of audit log entries. Existing (non-demo) data in the tenant is left completely untouched. Refuses to run again if demo data is already loaded — check the `demo_active` flag on the tenant.
- **Remove Demo Workspace** — deletes every row tagged `is_demo = True` for that tenant only, across all tables, and restores the tenant’s real branding from the backup taken just before demo data was loaded.

Both actions are gated to the `owner` role and scoped strictly to `current_user.tenant` — see `load_demo_data()` / `remove_demo_data()` in `app.py` for the implementation, and `SECURITY.md` for the safety guarantees (tenant isolation, no real data ever deleted, demo accounts have an unusable password).

Before using this on an existing deployment, run the migration in §4 above — Demo Mode will fail without it. -e

7. Frequently Asked Questions

Frequently Asked Questions

Licensing

Is GhostLeaks Enterprise Edition open source? No. It is proprietary commercial software licensed as source code. See `LICENSE.md` for exact terms.

Can I white-label it and sell it to my own clients? Yes — white-labeling and reselling breach-monitoring services built on the Software are explicitly permitted under `LICENSE.md` Section 1. What you may not do is redistribute or resell the original source code itself as your own product — see Section 2.

Can I modify the source code? Yes, for your own internal or commercial use, per `LICENSE.md` Section 1.2.

Can I publish the source code on GitHub or another public repository? No. This is explicitly prohibited under `LICENSE.md` Section 2.2, regardless of whether the repository is public or accessible to others by mistake.

Do I get free updates forever? No. This is a one-time purchase covering the version delivered at purchase. See `LICENSE.md` Section 7 and `SUPPORT.md` for what ongoing support options may separately exist.

Can I get a refund? Refund terms are not defined in `LICENSE.md` and depend on the seller’s own policy at time of sale. Ask via `SUPPORT.md` .

Deployment

Do I need Docker? No — Docker Compose is the recommended path, but a manual deployment (your own Python environment plus a database) is also supported. See `DEPLOYMENT.md` .

Do I need PostgreSQL, or can I use SQLite? SQLite works out of the box with zero configuration and is

fine for evaluation. PostgreSQL is recommended for production — SQLite is a single local file and isn't a good fit for most hosting setups long-term. See `DEPLOYMENT.md`.

Can multiple companies use one deployment? Yes — the application is multi-tenant. Each company that registers (or is invited via a workspace's invite link) gets an isolated tenant with its own data, branding, and domains. See `README.md`.

Does GhostLeaks host anything for me? No. You run the application on your own infrastructure (or your customer's). There is no seller-operated backend your deployment depends on. See `BUYER_GUIDE.md`.

Breach Data and Providers

Do I need my own breach-provider API key? Yes. GhostLeaks does not include or resell access to any breach-intelligence provider — you bring your own subscription and API key (BreachDirectory, HIBP, LeakCheck, or XposedOrNot are currently supported). See `README.md`.

What happens if a breach provider changes its API or shuts down? The provider abstraction layer (`providers/`) means switching or adding a provider doesn't require changing the rest of the application — but the seller does not control or guarantee any third-party provider's availability, pricing, or API stability. See `LICENSE.md` Section 4.

Does a "clean" scan result guarantee an email or domain has never been breached? No. A clean result means nothing was found in the currently configured provider(s)' data at the time of the scan — it is not a guarantee of zero exposure, especially for domain-wide scans without employee emails added. See `SECURITY.md` known limitations and `README.md`.

Can I use more than one breach provider at once? Yes — set `BREACH_PROVIDER` to a comma-separated list. Findings confirmed by more than one provider score with higher confidence. See `ARCHITECTURE.md`.

Data and Compliance

Where is my data stored? In the database you configure (`DATABASE_URL`) — either the bundled Docker Postgres container or an external database you point it at. The seller does not have access to or custody of your deployment's data.

Is this GDPR/HIPAA/[compliance framework] compliant? That depends entirely on how you deploy, configure, and operate it — the seller makes no compliance certification for the Software itself. See `LICENSE.md` Section 6.6. You are responsible for determining whether your specific deployment and use case meets your compliance obligations.

Does GhostLeaks store leaked passwords? No. Password checks use k-Anonymity, and every breach-checking code path returns only source name, confidence score, and status — never a password or credential value. See `SECURITY.md`.

Support

How do I get help? See `SUPPORT.md` for contact details and what to check before reaching out.

Is there a guaranteed response time? Not by default — see `SUPPORT.md` for what is and isn't covered without a separate support plan.

Where do I report a bug vs. a security vulnerability? Both go through the same contact in `SUPPORT.md`, but security issues should never be reported in a public channel — see `SECURITY.md`. -e

8. Selling and Positioning This Product

Buyer Guide

This guide is for someone who has just licensed GhostLeaks Enterprise Edition and is deciding how to deploy and position it. It assumes you've read `README.md` for the feature list and `LICENSE.md` for what you're legally permitted to do.

What you are actually getting

You are licensing **source code**, under the terms in `LICENSE.md` — not a hosted SaaS subscription, not a managed service, and not open-source software you can redistribute. Concretely, that means:

- You run it on infrastructure you control (or your customer's, if you're an MSP deploying into a client's environment).
- You bring your own accounts for every third-party service it uses: a breach-intelligence provider, a database, an email provider, and (if you charge customers directly through the product) a payment processor.
- There is no shared multi-customer SaaS instance run by the seller that your customers log into — each deployment is yours.

If your business model assumes a hosted, seller-operated backend, that is not what this license grants — re-read `LICENSE.md` Section 1 before committing to a go-to-market plan that assumes otherwise.

Before you sell anything

1. **Get real breach-provider access.** The default provider (BreachDirectory via RapidAPI) requires your own subscription and API key. If you want cross-provider confidence scoring (see `ARCHITECTURE.md` §4), set up more than one provider before launch, not after — confidence scores are capped low with a single provider, and that's visible to end users.
2. **Decide your domain-monitoring story honestly.** Domain-wide scanning is only as good as the employee emails you (or your customer) add per domain — see Domain Monitoring Workflow. Don't market "full domain breach coverage" without that step being part of your onboarding process for customers.
3. **Read `SECURITY.md` known limitations in full**, especially the rate-limiting and RBAC notes, before deciding whether the current state is production-ready for your specific customer base.
4. **Decide your own support and update policy** and put it in front of your customers before they buy from you — `LICENSE.md` explicitly does not promise ongoing updates as part of the base purchase.
5. **Set up a scheduled trigger** for `/send-breach-alerts` before promising customers "daily monitoring" — this doesn't run on its own; see `DEPLOYMENT.md` §6.

Positioning by buyer type

MSPs / cybersecurity agencies You're likely white-labeling this entirely and adding it as a line item to an existing security services contract. Your main integration points are branding (Workspace Settings) and the invite-link flow for onboarding client staff into their own isolated tenant.

Software agencies / SaaS companies You're more likely to use the REST API (see `API.md`) to embed breach checks into an existing product rather than presenting the GhostLeaks UI directly to end users. Plan for the fact that the current API surface is intentionally small (email check, password check, risk score) — extending it means modifying `app.py` directly, since there's no plugin system for API routes.

Internal security teams You're likely the only tenant on the deployment. Multi-tenancy and white-labeling are mostly irrelevant to you — focus on the domain-monitoring workflow and getting employee emails populated accurately, since that's what actually drives useful results for your organization.

What this product does not include

Being direct about this now saves a support conversation later:

- A managed hosting platform. You (or your hosting provider) operate the infrastructure.
- A guarantee that any specific breach-data provider will remain available, priced the same, or behave the same over time — see `LICENSE.md` Section 4.
- A built-in background job scheduler for the daily alert job — you supply the trigger.
- A public plugin/extension marketplace — extending providers or features means editing the source directly, within your license terms.
- An automated test suite as of this writing — see `ARCHITECTURE.md` known gaps.

Getting Help

See `SUPPORT.md` for how to reach the seller, and `FAQ.md` for answers to common buyer questions before reaching out. -e

9. License Summary

GhostLeaks Enterprise Edition — Commercial License Agreement

Version: 1.0 **Effective Date:** Date of purchase, as recorded in the applicable order confirmation or invoice. **Licensor:** Ritesh, operating as GhostLeaks ("Licensor", "Seller") **Product:** GhostLeaks Enterprise Edition (the "Software")

> This is a template commercial license agreement, not a substitute for legal advice. It has not been reviewed by a licensed attorney and does not constitute legal advice. Before using this document to sell or license commercial software, have it reviewed by a qualified attorney in your jurisdiction, particularly regarding liability limitations, consumer-protection law, and export/data-protection regulations applicable to your buyers.

---

1. Grant of License

This is not open-source software. GhostLeaks Enterprise Edition is proprietary, commercially licensed software distributed as source code. By purchasing, downloading, installing, or using the Software, the purchaser ("Licensee", "Buyer", "you") agrees to be bound by the terms of this Agreement.

Subject to full payment of all applicable fees and continued compliance with this Agreement, Licensor grants Licensee a non-exclusive, non-transferable (except as permitted under Section 8), worldwide license to:

1. **Deploy** the Software on infrastructure owned, leased, or controlled by Licensee, including cloud infrastructure operated on Licensee's behalf.
2. **Modify** the source code for Licensee's own internal or commercial purposes, including customization, bug fixes, and feature additions.

3. **White-label** the Software, including replacing product name, branding, logos, and visual design, for presentation to Licensee's own customers or internal users.
4. **Connect Licensee's own third-party service accounts** to the Software, including breach-intelligence providers, payment processors, email delivery services, and databases.
5. **Use Licensee's own database infrastructure**, whether self-hosted or managed by a third party.
6. **Deploy for unlimited internal users** within the organization(s) covered by the applicable license tier, as defined in the order/purchase documentation associated with this license.
7. **Offer breach-monitoring services to Licensee's own clients** using the Software as the underlying platform, including charging fees for such services.

This license covers **one deployment/licensed organization** unless otherwise specified in a separate written agreement or order confirmation between Licensor and Licensee.

---

2. Restrictions

Licensee shall **not**:

1. Redistribute, publish, sublicense, or otherwise make available the original source code of the Software, in whole or in part, to any third party, except as strictly necessary to operate the Software (e.g., to Licensee's own hosting provider or employees under confidentiality obligations).
2. Publish the source code, or any substantial portion of it, in any public repository, forum, code-sharing platform, or similar venue (including but not limited to GitHub, GitLab, Bitbucket, or public package registries).
3. Resell, relicense, or distribute the original, unmodified (or superficially modified) Software as if it were Licensee's own independently developed product, separate from Licensee's own value-added service offering described in Section 1.7.
4. Remove, obscure, or alter any copyright notice, proprietary notice, or license notice embedded in the Software's source code, documentation, or user interface, except where such removal is an explicit, intended part of permitted white-labeling under Section 1.3 and does not involve claiming original authorship under Section 3.
5. Claim authorship of the original GhostLeaks Enterprise Edition project, its architecture, or its original source code. Licensee may describe itself as the operator, reseller, or provider of a branded service built on the Software, but may not claim to have originally authored the underlying platform.

Any breach of this Section constitutes a material breach of this Agreement and may result in immediate termination of the license under Section 9, without prejudice to any other remedies available to Licensor.

---

3. Intellectual Property

Licensor retains all right, title, and interest in and to the original Software, including all copyrights, trade secrets, and other intellectual property rights therein, whether or not registered. This Agreement does not transfer ownership of the Software to Licensee.

Modifications made by Licensee to the source code are owned by Licensee, subject to Licensor's continuing ownership of the unmodified, underlying Software that such modifications are built upon.

---

4. Third-Party Services

The Software is designed to integrate with third-party services, which may include (depending on configuration): breach-intelligence data providers, payment processors, transactional email providers, and database hosting services.

1. **Licensee is solely responsible** for obtaining, configuring, and maintaining its own accounts, subscriptions, and API keys for any third-party service the Software connects to.

2. **Licensee is solely responsible** for complying with the terms of service, acceptable use policies, and pricing terms of each third-party service it chooses to use.
3. **Licensor does not operate, control, warrant, or guarantee the availability, accuracy, or continued existence of any third-party service.** Third-party providers may change their APIs, pricing, terms of service, or discontinue service entirely, at any time and without notice to Licensor.
4. **Licensor is not responsible** for any changes made by third-party providers that affect the Software's functionality, including but not limited to API changes, rate-limit changes, deprecations, or service discontinuations.
5. Licensee is responsible for its own hosting infrastructure and database, whether self-managed or provided by a third party, and for maintaining appropriate backups, security configurations, and access controls for that infrastructure.

---

5. Disclaimer of Warranty

THE SOFTWARE IS PROVIDED "AS IS" AND "AS AVAILABLE," WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, OR STATUTORY, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE, NON-INFRINGEMENT, OR ANY WARRANTY ARISING FROM COURSE OF DEALING OR USAGE OF TRADE.

Without limiting the foregoing, Licensor does not warrant that:

- The Software will be uninterrupted, error-free, or free of security vulnerabilities.
- Any breach data returned by third-party providers is complete, accurate, or up to date.
- The Software will meet Licensee's specific business, security, or compliance requirements.
- Defects will be corrected, or that the Software will continue to be compatible with third-party services over time.

---

6. Limitation of Liability

To the maximum extent permitted by applicable law, in no event shall Licensor be liable for:

1. **Data loss** of any kind, including loss of Licensee's or Licensee's end-customers' data.
2. **Security incidents**, including unauthorized access, data breaches, or misuse of the Software by Licensee, Licensee's employees, or Licensee's end-customers.
3. **Changes made by third-party API or service providers**, including breach-intelligence providers, payment processors, or email providers, that affect the Software's functionality or availability.
4. **Third-party outages** or unavailability of any service the Software integrates with.
5. **Business losses**, including lost profits, lost revenue, lost business opportunities, or reputational harm, whether arising from use of the Software or inability to use it.
6. **Compliance failures**, including but not limited to failure to comply with data-protection law (e.g., GDPR, CCPA, or similar regimes), industry-specific regulation, or contractual obligations Licensee has with its own customers. Licensee is solely responsible for determining whether its use of the Software, and any service it builds on top of the Software, complies with laws and regulations applicable to Licensee and Licensee's customers.

In no event shall Licensor's total aggregate liability arising out of or related to this Agreement exceed the amount actually paid by Licensee to Licensor for the Software license.

---

7. Updates and Support

This license is sold as a **one-time purchase** granting perpetual use of the version of the Software delivered at the time of purchase, unless otherwise stated in a separate written agreement.

1. **No lifetime updates are promised or implied.** Licensor may, at its discretion, offer future versions,

bug fixes, or feature updates, but is under no obligation to provide them as part of this license.
2. Licensor may offer optional, separately priced support and/or update plans (e.g., annual maintenance, priority support, or upgrade subscriptions). Availability, scope, and pricing of any such plans are governed by the terms in effect at the time such a plan is purchased and are not part of this Agreement.
3. Absent a separate support agreement, Licensor has no obligation to provide bug fixes, security patches, upgrade assistance, or technical support of any kind.

See `SUPPORT.md` for current support channels and `ROADMAP.md` for planned (non-guaranteed) future work.

---

8. Transfer and Assignment

This license is granted to the named Licensee (the purchasing individual or organization) and is non-transferable except:

- In connection with a bona fide sale of substantially all of Licensee's business or assets to a successor entity, provided the successor agrees in writing to be bound by this Agreement; or
- With Licensor's prior written consent.

---

9. Termination

Licensor may terminate this license immediately upon written notice if Licensee materially breaches this Agreement, including but not limited to the restrictions in Section 2, and fails to cure such breach within a reasonable cure period after receiving notice, where curable.

Upon termination:

- Licensee must cease all use of the Software and destroy or return all copies of the source code in its possession.
- Sections 2 (Restrictions), 3 (Intellectual Property), 5 (Disclaimer of Warranty), 6 (Limitation of Liability), and this Section 9 survive termination.
- Termination does not entitle Licensee to a refund of amounts already paid, except as required by applicable law or as separately agreed in writing.

---

10. Governing Law

This Agreement's governing law and jurisdiction are to be specified by Licensor. This template intentionally does not select one — consult an attorney to draft appropriate governing-law and dispute-resolution terms for your business location and your buyers' locations.

---

11. Entire Agreement

This Agreement, together with any order confirmation, invoice, or separate written agreement referencing it, constitutes the entire agreement between Licensor and Licensee regarding the Software and supersedes all prior or contemporaneous understandings regarding its subject matter. If any provision of this Agreement is held unenforceable, the remaining provisions remain in full force and effect.

Contact

For licensing questions, contact: Ritesh — ritdmdkkd@gmail.com-e

10. Version History

Changelog

All notable changes to GhostLeaks Enterprise Edition are documented here. Format loosely follows Keep a Changelog; version numbers below are illustrative and should be aligned with your own release/versioning policy before publishing to buyers.

> Dates reflect when each change was made in the source history available at the time this file was written.

[1.0.2] — 2026-08-12

Payment provider swap and self-service GDPR tooling. No changes to tenant isolation, breach-checking logic, or the domain-verification flow.

Changed

- **Payment processor switched from Razorpay to Stripe.** `/create-checkout-session` replaces the old `/create-order` + `/verify-payment` pair, using Stripe Checkout (hosted payment page) rather than a client-side JS widget — simpler integration surface, no card data ever touches this server. Activation now happens via a `checkout.session.completed` webhook (`/webhooks/stripe`) rather than a client-reported "payment succeeded" call, which is more resistant to a client just skipping the success callback. `User.razorpay_payment_id` is renamed to `User.stripe_payment_id` ; see `DEPLOYMENT.md` for the migration statement on an existing database. Order-ownership binding and payment-ID uniqueness (the two protections added in 1.0.1) are preserved under the new provider.
- **Price and currency are now environment-configurable** (`PLAN_AMOUNT_CENTS` , `PLAN_CURRENCY` , `PLAN_PRICE_LABEL`) instead of a hardcoded INR amount — a deployment can price in USD, EUR, or any Stripe-supported currency without touching code.
- **`/pricing` is now linked from the landing page nav and the dashboard nav** (as "Upgrade", shown only to unpaid users) — it previously existed as a route with no link pointing to it anywhere in the UI.

Added

- **Self-service data export** (`/settings/export-data`) — downloads the requesting user's own account info, scan history, credential findings, and audit-log entries as JSON. Never includes password hash, API key hash, or another user's data.
- **Self-service account deletion** (`/settings/delete-account`) — password-confirmed. Refuses to delete a workspace `owner` while other members still exist (would orphan the workspace), and refuses to delete a user who has added a monitored domain (would leave a `DomainMonitor` row with a dangling `NOT NULL` foreign key) — both cases return a clear message telling the user what to do first. Scan history (exclusively the user's own) is hard-deleted; credential findings and audit-log entries (shared organizational security data) are kept but stripped of the personal link, rather than deleted outright.
- Both are linked from `/settings` under a new "Your Data" section.

Fixed

- `requirements.txt` now pins `psycopg[binary]` (psycopg 3) instead of `psycopg2-binary` , and caps `setuptools<81` — `psycopg2-binary` 's compiled extension is not ABI-compatible with newer CPython builds, and `pkg_resources` (needed by the `razorpay` package's dependency chain, and still referenced by some transitive dependencies) was removed from `setuptools` 81+.
- Added `runtime.txt` pinning `python-3.11.9` — without a version pin, a native-buildpack host (Render, Railway, etc.) defaults to its newest available Python, which this app was not tested against and which broke several C-extension dependencies.

- Postgres connection URLs (`postgres://` and `postgresql://`) are now rewritten to `postgresql+psycopg://` so SQLAlchemy actually loads the psycopg 3 driver instead of falling through to the (now-uninstalled) psycopg2 dialect.
- Added `SQLALCHEMY_ENGINE_OPTIONS` (`prepare_threshold: None`) when running on Postgres — psycopg 3's server-side prepared statements can collide under a transaction-mode connection pooler (e.g. Supabase's pooled connection string, PgBouncer), producing an intermittent `DuplicatePreparedStatement` error. This disables server-side prepare so the driver behaves correctly regardless of which pooling mode the database sits behind.
- Added `.gitignore` (previously absent) so `.env` , `__pycache__/` , and local `.db` files can no longer be committed by accident.

[1.0.1] — 2026-08-05

Fixes from a second, independent pre-launch audit. All four claims were individually verified against the actual code before being fixed — one (API key storage) was corrected in scope; see the note below.

Security

- **Fixed a payment-ownership gap that could let one account activate premium using another account's payment.** `/verify-payment` verified the Razorpay signature (proving `order_id` / `payment_id` / `signature` are mutually consistent) but never checked that the order belonged to the account trying to redeem it. Orders are now bound to the requesting user at creation (`User.pending_order_id`), checked before activation, and `razorpay_payment_id` is now a unique column so the same payment can't activate two accounts.
- **API keys are now hashed rather than stored in plaintext.** New and regenerated keys are stored as a SHA-256 hash; the raw value is shown once at generation and can't be retrieved afterward. Keys issued before this change keep working via a backward-compatible lookup — no existing integration breaks.
  - **Scope correction:** the original report described this as "third-party API keys stored in plaintext." That's not accurate — no third-party key (RapidAPI, HIBP, LeakCheck, XposedOrNot, Brevo, Razorpay) is ever stored in the database; all of those are read from environment variables only, which is the correct practice. The actual plaintext-stored value was `User.api_key` — the key *this application issues* to its own users for REST API access. That's what this fix addresses.
- **Rate-limiter storage is now configurable.** Set `REDIS_URL` to back `Flask-Limiter` with Redis instead of the default in-memory store, so limits are accurate across multiple Gunicorn workers rather than effectively multiplied by worker count. Off by default — no existing deployment is affected until `REDIS_URL` is explicitly set. An optional `redis` service was added to `docker-compose.yml` behind the `with-redis` Compose profile, so a plain `docker compose up -d` behaves exactly as before.
- **The Docker image no longer runs as root.** `Dockerfile` had no `USER` instruction at all (the implicit default). Added a dedicated `appuser` and switched to it before the app starts. Nothing in the application needs root — it binds to an unprivileged port and writes no files to disk.

[1.0.0] — 2026-08-04

First launch-ready release. This version focuses on stability, security hardening, and deployment readiness — no new user-facing features were added; see `ROADMAP.md` for what was deliberately deferred to v1.1/v2.0 to avoid regression risk this close to release.

Added

- **Baseline security headers** on every response (`X-Content-Type-Options` , `X-Frame-Options` , `Referrer-Policy` , `Permissions-Policy` , `Content-Security-Policy` , and `Strict-Transport-Security` when served over HTTPS).
- **`/health` endpoint** — checks app liveness and database connectivity, for load balancers and uptime monitors.

- `.github/workflows/ci.yml` — basic CI: compiles `app.py` / `providers/`, validates every template parses, validates `docker-compose.yml`. Not a behavioral test suite (none exists yet — see `ROADMAP.md`), but catches syntax/import breakage on every push.
- Dependencies in `requirements.txt` are now **pinned to specific versions** for reproducible builds, instead of unpinned.

Fixed

- Constant-time comparison (`hmac.compare_digest`) for the scheduled-alert shared-secret check, replacing a plain `!=` comparison.
- Documentation cleanup: `ARCHITECTURE.md`'s known-gaps list still described the `admin` role as unreachable, which was fixed in an earlier change (role promotion via Workspace Settings) — the doc just hadn't been updated to match. Corrected.

Documentation

- `ROADMAP.md` restructured into explicit **v1.1** (near-term: webhook signing, account deletion/export, invite-link expiry, owner succession, migrations, shared rate limiting, background job queue, test suite, `datetime.utcnow` migration) and **v2.0** (larger work: blueprint refactor, SSO/SAML, native domain-search provider, row-level security, usage metering, PSA/SIEM integrations) sections, so buyers and future contributors have a clear, honest picture of what's intentionally not in v1.0.0 and why.
- Verified every environment variable read by `app.py` is documented in both `.env.example` and `README.md`'s variable table, and vice versa — no undocumented or stale entries found.

[2.7.0] — 2026-08-02

Fixes from a full static security/correctness audit of the codebase. All items were individually verified against the actual code before being fixed (not applied blindly from the audit report).

Security

- **Fixed reflected XSS in bulk-scan results, single email-check results, and the exposure timeline.** User-supplied content (an uploaded CSV's email column has no format validation beyond requiring an `@`, so it can contain arbitrary HTML/JS) was interpolated directly into `innerHTML` in `dashboard.html` and `timeline.html`. All dynamic values (`r.email`, `b.name`, `e.email`, `e.source`, `e.status`, error messages) are now passed through an `escapeHtml()` helper before insertion.
- **Fixed the REST API incorrectly blocking deployments configured for a non-BreachDirectory provider.** `/api/v1/check-email` checked `if not RAPIDAPI_KEY` specifically instead of checking whether *any* provider was active — a deployment configured with only `HIBP_API_KEY` (and no `RAPIDAPI_KEY`, which is correct if BreachDirectory isn't being used) was incorrectly told breach checking wasn't configured. Now checks `BREACH_PROVIDERS` generically.
- **Fixed `known_breaches` using substring matching instead of exact matching** in the scheduled alert job — comparing a new source name against a raw comma-joined string with Python's `in` operator can false-match (e.g. "App" inside "MyApp") or false-miss depending on naming overlap. Now compared against a proper set of exact tokens.
- **Added rate limiting to `/send-breach-alerts`.** It was previously protected only by the `ALERT_SECRET` header check with no rate limit — if that secret ever leaked, it had no limit on how often it could be triggered.
- **Excluded demo accounts from the scheduled alert job.** It previously selected all `email_verified = True` users with no `is_demo` exclusion, so demo accounts (which are `email_verified = True` by design) were included — wasting provider quota and attempting to email addresses at a domain that doesn't receive mail.
- **URL-encoded the email parameter in the HIBP, LeakCheck, and XposedOrNot provider requests.** These built the request URL with an f-string directly embedding the raw email in the path; an email containing `+`, `#`, or other URL-special characters could produce a malformed or wrong request. (BreachDirectory was unaffected — it already passed the email as a `requests` query parameter, which

encodes automatically.)
- **`upsert_finding()` could return `None` after a rare concurrent-insert race**, and two call sites (`check_email()`, `/api/v1/check-email`) accessed `.source_name` on the result unconditionally, which would raise `AttributeError` in that edge case. Added a defensive retry plus explicit `None` filtering at both call sites.

Fixed

- `check-password` (web and API) previously treated a failed provider check (`None`) the same as a confirmed-safe result (`False`) — `'leaked' if found else 'safe'` is true for both. A failed check now returns a distinct `CHECK_FAILED` error instead of silently reporting "safe."
- `log_audit()` would raise an uncaught `IntegrityError` (crashing the caller's request) if called with no actor and no authenticated user, since `AuditLog.tenant_id` is `NOT NULL`. Now logs a warning and skips the entry instead of crashing.
- Registration accepted any non-empty string as an email address with no format check. Added a basic regex validation.
- The auto-generated tenant slug only replaced spaces with hyphens, so a company name containing `/`, `&`, `#`, etc. produced a slug with characters that break or confuse the `?tenant_slug=` invite-link query parameter. Now sanitized to letters, numbers, and hyphens only.
- Workspace branding's primary-color field only checked for a leading `#` and length 7, not that the remaining characters were valid hex digits — an invalid value passed validation and later crashed PDF report generation inside ReportLab's `HexColor()`. Now validated with a proper hex regex.
- Adding "employee emails" to a verified domain didn't check the email actually belonged to that domain — any address could be added under any monitored domain. Now rejected if the email's domain doesn't match.
- Three `request.get_json()` calls (`check_email()`, `verify_payment()`, `download_report()`) didn't use `silent=True`, so a malformed/empty request body produced Flask's generic 400 page instead of the app's own JSON error responses. Made consistent with the rest of the codebase.
- Documented (in `docker-compose.yml` and `.env.example`) that `POSTGRES_PASSWORD` cannot contain URL-special characters (`@ : / # %`) — Docker Compose builds it directly into a `postgresql://` connection string and cannot URL-encode it automatically.
- Removed a stray duplicate `templates/footer.html` left over from an earlier manual upload — the real, live file is `templates/partials/footer.html`; the duplicate was never included by any template but was confusing to find in the repo.

[2.6.2] — 2026-08-01

Added

- **Configurable default branding for pre-login pages.** The landing page, login, register, and other public pages aren't tied to any single tenant, so per-tenant white-label branding never applied to them — they always showed "GhostLeaks" regardless of any workspace's branding settings. New `DEFAULT_BRAND_NAME`, `DEFAULT_BRAND_COLOR`, and `DEFAULT_BRAND_LOGO` environment variables now control this instead of a hardcoded value, so a deployment dedicated to a single company can be fully rebranded via configuration — no code edits needed.

Fixed

- Every public-facing template (`landing.html`, `login.html`, `register.html`, `forgot_password.html`, `reset_password.html`, `pricing.html`, `about.html`, `documentation.html`, `privacy.html`, `terms.html`, `contact.html`, and the shared footer partial) had the brand name and accent color hardcoded directly in the template rather than reading the `brand` context variable — meaning even the new `DEFAULT_BRAND_*` variables above would have had no visible effect until these were fixed. Page `<title>` tags were also hardcoded and are now dynamic.

[2.6.1] — 2026-07-29

Fixed

- **Demo Mode failed with a 500 error for any second tenant on the same deployment.** Demo user accounts were created with fixed email addresses (e.g. `admin@acme-demo.com`); `User.email` has a global unique constraint across the whole app, not per-tenant, so a second workspace loading demo data collided with the first workspace's demo users and crashed instead of failing gracefully. Demo emails are now built per-tenant as `{name}+{tenant-slug}@acme-demo.com`, using each tenant's own (already-unique) slug, so this can never collide across workspaces again.

[2.6.0] — 2026-07-28

Added

- **Demo Mode.** Workspace owners can now load realistic sample data into their own tenant from Workspace Settings — a demo company, five demo users, a pre-verified demo domain, credential findings across multiple sources and every status value, weeks of scan history, and matching audit log activity — for demonstrations, screenshots, and buyer evaluation. Every row created this way is tagged `is_demo = True` and scoped to that tenant; "Remove Demo Workspace" deletes exactly those rows and restores the tenant's real branding from a backup taken before demo branding was applied. Restricted to the `owner` role; cannot affect any other tenant. See `SECURITY.md#demo-mode` and `DEPLOYMENT.md#9-demo-mode`.
- New `is_demo` column on `User`, `ScanHistory`, `DomainMonitor`, `DomainEmployeeEmail`, `CredentialFinding`, and `AuditLog`; new `demo_active` / `demo_branding_backup` columns on `Tenant`. **Existing deployments must run a manual migration** for these columns before Demo Mode will work — see `DEPLOYMENT.md`.

[2.5.0] — 2026-07-28

Changed

- **Removed the payment gate on core features.** Email check, bulk CSV check, PDF report download, and the REST API previously required `is_paid = True` on the account (set after a successful Razorpay payment); this check has been removed from all four, so every account can use them regardless of payment status. Password check was already free and is unaffected. The Razorpay payment flow (`/pricing`, `/create-order`, `/verify-payment`) still exists in the codebase and still marks an account as paid on success, but nothing currently checks that flag — re-introducing a paywall (for this or a different tier of features) means adding the check back at the relevant route(s).

[2.4.0] — 2026-07-28

Added

- **Admin role promotion/demotion.** The workspace owner can now promote a member to `admin` or demote an admin back to `member` from Workspace Settings (`/admin/tenant-settings`). Previously, the `admin` role was recognized by permission checks but had no assignable path — only `owner` and `member` were reachable. An owner cannot change their own role through this control, and only the owner can change another member's role; every change is recorded in the audit log.

[2.3.0] — 2026-07-26

Changed

- **Domain monitoring now scans individual employee emails.** Previously, a verified domain was scanned by passing the bare domain name as a search term to the breach provider — which rarely returned meaningful results, since breach providers index individual emails, not domain names. Domains can now have employee emails added after verification; each tracked email is checked individually and results are rolled up under the domain. Domains with no employee emails added yet still fall back to the old domain-string search, now clearly labeled as a best-effort fallback rather than the primary mechanism.
- Added a new `DomainEmployeeEmail` record type, scoped to a verified `DomainMonitor`.

Added

- Routes to add and remove employee emails tracked under a domain.

[2.2.0] — 2026-07-25

Changed

- Full front-end visual redesign across every page (login, register, dashboard, findings, timeline, settings, workspace, audit log, pricing, landing) using a single shared design system (`static/gl-design-system.css`).
- No backend, routing, or business-logic changes were made as part of this release — it was scoped to presentation only.

[2.1.0] — 2026-07-24

Added

- **Provider abstraction layer** (`providers/`). Breach-data source integrations (previously a single inline function for BreachDirectory) were extracted behind a common interface (`base_provider.py`), with BreachDirectory, HIBP, LeakCheck, and XposedOrNot implemented against it.
- `BREACH_PROVIDER` environment variable to select one or more active providers without code changes.

Changed

- `aggregate_breach_sources()` now iterates configured provider instances instead of a hardcoded single provider call. Behavior for existing BreachDirectory-only deployments is unchanged by default.

[2.0.0] — 2026-07-22

This release is a substantial rebuild focused on a credential-exposure security fix and the addition of enterprise features (multi-tenancy, white-labeling, domain monitoring, audit logs).

Security

- **Fixed:** the previous version’s email-check logic returned the actual leaked password for any email address submitted — including other people’s emails, with no ownership verification, up to 100 at a time via bulk CSV upload. Every breach-checking code path now returns only source name, confidence score, and status; no password, hash, or credential value is ever returned. This is enforced at the provider layer, not only in the UI or API response formatting.

Added

- **Credential status lifecycle.** Findings now move through `active → rotated / historical / accepted_risk / resolved` instead of a flat leaked/safe flag.
- **Domain monitoring**, gated behind DNS TXT record ownership verification (see Known Limitations in `SECURITY.md` for the domain-scan honesty note that predates the 2.3.0 employee-email fix above).
- **Exposure timeline** (`/timeline`) visualizing when each exposure was discovered and when its status changed.
- **Confidence scoring and source correlation** — findings confirmed by more than one active provider score higher.
- **Audit log** (`/admin/audit-logs`), separate from per-user scan history.
- **Multi-tenant architecture** — `Tenant` model, invite-link onboarding (`/register?tenant_slug=...`), tenant isolation at the query layer.
- **White-label branding** — per-tenant name, color, and logo applied across dashboard, PDF reports, and email.

Changed

- Removed prior “dark web” / alarmist branding language in favor of a neutral, professional tone.

Earlier history

2.0.0 is the first documented release in this changelog.